COMPLETE STUDY NOTES

Fundamentals of Artificial Intelligence

Clear explanations, worked examples and memory hooks

What is inside

- Introduction to AI and its history
- Intelligent agents, PEAS, environments and agent types
- Uninformed search: BFS, UCS, DFS, DLS, IDS, bidirectional
- Informed search: Greedy best-first, A*, RBFS, heuristics
- Special lesson: GBFS with Manhattan distance
- Local search: hill climbing, annealing, beam, genetic algorithms
- Adversarial search: minimax and alpha-beta pruning
- Revision sheet with likely exam questions

Contents

How to Use These Notes . . . 4

Part 1: Getting to Know Artificial Intelligence . . . 5

1.1 AI in Fiction and AI in Real Life . . . 5
1.2 What Is Intelligence, and What Makes It Artificial? . . . 5
1.3 The Four Ways to Define AI . . . 6
1.4 The Fields That Built AI . . . 8
1.5 A Short History of AI . . . 8
1.6 AI Today and Where It Is Published . . . 9

Part 2: Intelligent Agents . . . 10

2.1 Agents and Environments . . . 10
2.2 The Vacuum Cleaner World . . . 11
2.3 Rationality: Doing the Right Thing . . . 12
2.4 PEAS: Describing the Task Before You Build the Agent . . . 12
2.5 Types of Environment . . . 13
2.6 Agent Functions and Agent Programs . . . 15
2.7 The Five Kinds of Agent . . . 16
2.8 Properties of Agents and a Quick Summary . . . 17

Part 3: Uninformed Search: Finding a Path With No Hints . . . 18

3.1 The Big Idea of Searching . . . 18
3.2 Breadth-First Search (BFS) . . . 19
3.3 Uniform-Cost Search (UCS) . . . 19
3.4 Depth-First Search (DFS) . . . 20
3.5 Depth-Limited Search (DLS) . . . 20
3.6 Iterative Deepening Search (IDS) . . . 20
3.7 Avoiding Repeated States . . . 21
3.8 Bidirectional Search . . . 21
3.9 Uninformed Search at a Glance . . . 21

Part 4: Informed Search: Search With a Compass . . . 22

4.1 Best-First Search and the Heuristic Function . . . 22
4.2 Greedy Best-First Search (GBFS) . . . 23

4.3 A* Search 24
4.4 Recursive Best-First Search (RBFS), the Memory-Saving Version 26
4.5 Designing Good Heuristics (the 8-Puzzle) 26
4.6 Informed Search Summary 28

Part 5: Special Lesson: Greedy Best-First Search with Manhattan Distance 29

5.1 Manhattan Distance From Scratch 29
5.2 Manhattan Distance on the 8-Puzzle 29
5.3 The GBFS Recipe 30
5.4 Worked Example A: GBFS on an 8-Puzzle 30
5.5 Worked Example B: GBFS on a Grid With Walls 31
5.6 GBFS With Manhattan Distance in Python 32
5.7 Common Mistakes and an Exam Checklist 33

Part 6: Beyond Classical Search: Local Search 34

6.1 Why We Need Something Beyond Classical Search 34
6.2 Hill-Climbing Search 35
6.3 Simulated Annealing 36
6.4 Local Beam Search 37
6.5 Genetic Algorithms (GA) 37

Part 7: Adversarial Search: Playing Games 40

7.1 Games Versus Ordinary Search 40
7.2 The Minimax Algorithm 40
7.3 Alpha-Beta Pruning 41
7.4 Evaluation Functions and Real Games 42

Part 8: Last-Day Revision Sheet 44

8.1 Search Algorithms Side by Side 44
8.2 Formulas to Memorise 44
8.3 Mnemonics Collected in One Place 45
8.4 Likely Exam Questions With Short Answers 45

How to Use These Notes

These notes cover every topic in your seven files: the AI introduction, the two intelligent agent decks, uninformed search, informed search, beyond classical search (local search) and adversarial search. On top of that there is a full extra lesson on **Greedy Best-First Search with Manhattan distance**, worked step by step. Each topic follows the same rhythm so your brain learns the pattern: watch, read, understand the key idea, then lock it in with a hook.

Watch First

A blue box at the start of a topic. It gives you Gate Smashers YouTube links to get the big picture before reading. Each link opens a YouTube search for that topic on the Gate Smashers channel, so you can pick the exact video that fits.

Concept Box

A green box. This is where an important idea gets slowed right down and explained in plain words with a small example. In the Intelligent Agents slides some bullet points carry a small coloured marker, and every one of those marked ideas has its own green box here. I also added green boxes for the other key ideas in the rest of the slides.

Memory Hook

An amber box with a short trick, a mnemonic or a one-line summary you can recall under exam pressure.

A good order to study in

- Read Parts 1 and 2 first. They are the vocabulary that everything else depends on.
- Read Part 3 (uninformed search) and Part 4 (informed search) together, because informed search is simply uninformed search with a guess about the future added.
- Do the special GBFS and Manhattan lesson in Part 5 with a pencil and paper. Redo the worked examples without looking.
- Finish with Parts 6 and 7, then use Part 8 as your last-day revision sheet.

Study Tip

Do not just read. After every Concept Box, close the page and explain the idea out loud in your own words. If you can teach it to a friend in two sentences, you know it.

PART 1

Getting to Know Artificial Intelligence

What AI is, how people define it, where it came from and what it can do today.

1.1 AI in Fiction and AI in Real Life

Watch First

Start with one of these, then come back and read the notes. Links to the Uninformed vs Informed and BFS lectures open the video directly. The others open a Gate Smashers search for that exact lecture title, so tap the top result.

- Gate Smashers: What is Artificial Intelligence | Learn AI with Real Life Examples

For almost a century, films have imagined machines that think. Many of those fantasies have quietly become real products.

AI in fiction (the films on the slides)

- **Metropolis** (1927, Fritz Lang), **2001: A Space Odyssey** (1968, Stanley Kubrick), **Star Wars** (1977, George Lucas)
- **Blade Runner** (1982, Ridley Scott), **Terminator** (1984, James Cameron)
- **A.I. Artificial Intelligence** (2001, Steven Spielberg), **I, Robot** (2004, Alex Proyas)
- **Wall-E** (2008, Andrew Stanton), **Ex Machina** (2015, Alex Garland), **Blade Runner 2049** (2017, Denis Villeneuve)

AI in reality

- Self-driving cars, autonomous vacuum cleaners (iRobot), soccer-playing robots (Sony)
- Acrobatic humanoid robots (Boston Dynamics), autonomous trading bots
- Voice assistants (Amazon Alexa), image and face recognition tools
- World-champion beating chess and Go players, and AI fighter pilots

Memory Hook

Fiction is the preview, reality is the release. Every film on the list has a real-world cousin on the list below it.

1.2 What Is Intelligence, and What Makes It Artificial?

Watch First

Start with one of these, then come back and read the notes. Links to the Uninformed vs Informed and BFS lectures open the video directly. The others open a Gate Smashers search for that exact lecture title, so tap the top result.

- Gate Smashers: What is Artificial Intelligence | Learn AI with Real Life Examples

Intelligence is a long list of abilities: logic, understanding, self-awareness, learning, reasoning, planning, creativity, critical thinking and problem solving. The slides boil it down to one sentence: the ability to **perceive or infer information, keep it as knowledge, and use it for adaptive behaviour** in an environment.

Concept Box: The puzzle example, step by step

The slides show a picture puzzle and ask for the missing piece (the answer was option B, a column-by-column clockwise rotation). Look at what your own brain did:

- **Perceive:** you read and recognised the figures.
- **Retain:** you kept the patterns and the candidate answers in memory.
- **Infer:** you worked out the hidden rule and chose the solution.

Those three verbs (perceive, retain, infer) are exactly what intelligence means. An AI system has to do the same three things with sensors, memory and reasoning.

Intelligence vs Artificial Intelligence. Studying intelligence means trying to *understand* how a small handful of matter (a brain) can perceive, predict and manipulate a world far bigger than itself. It draws on logic, psychology, cognitive science, neuroscience, philosophy, ethics and linguistics. **AI goes one step further: it tries to *build* intelligent entities.** So it also uses mathematics, computer science, engineering, economics, control theory and cybernetics, and electronics.

1.3 The Four Ways to Define AI

Watch First

Start with one of these, then come back and read the notes. Links to the Uninformed vs Informed and BFS lectures open the video directly. The others open a Gate Smashers search for that exact lecture title, so tap the top result.

- Gate Smashers: Turing Test and approaches of AI
- Gate Smashers: Intelligent agents and rational thinking

There is no single definition, because people disagree on two things: should AI *think* or *behave*, and should success be measured against *humans* or against an *ideal* (rationality)? Two choices on two axes give four approaches.

Thinking Humanly **Cognitive modelling** How do humans think? Build a theory of the mind.	**Thinking Rationally** **Laws of thought** Correct reasoning using logic.
Acting Humanly **The Turing Test** Behave so well that people cannot tell you are a machine.	**Acting Rationally** **The rational agent** Do the right thing to get the best outcome.

The four approaches: the top row is about thought processes, the bottom row is about behaviour. The left column measures against humans, the right column against an ideal.

Thinking humanly: the cognitive modelling approach

- Problem: how do humans actually think? Idea: build a theory of the mind and express it as a program.
- Example: Newell and Simon's **General Problem Solver (1961)**.
- This needs scientific theories of brain activity. The field that studies this is **Cognitive Science**, which mixes computer models from AI with experiments from psychology.
- AI and Cognitive Science are now separate. **AI** wants an algorithm that performs well on a task. **Cognitive Science** wants a good model of human performance. They still help each other, for example in computer

vision.

Acting humanly: the Turing Test

Alan Turing proposed this in his 1950 paper *Computing Machinery and Intelligence*, also called the **Imitation Game**. A human, a computer and an interrogator sit in different rooms. The interrogator chats with both and must decide which is the machine. If the computer fools the interrogator, it passes. In this view "behaves intelligently" means "behaves humanly".

> **Concept Box: What a machine needs to pass the Turing Test**
>
> Imagine building a chatbot that must fool a judge. You would need six abilities:
>
> - **Natural language processing** to talk in English (or another language).
> - **Knowledge representation** to store what it knows or hears.
> - **Automated reasoning** to answer questions and draw new conclusions.
> - **Machine learning** to adapt to new situations and spot patterns.
>
> The **Total Turing Test** adds a physical body, so the judge can also hand it objects. That needs two more:
>
> - **Computer vision** to see objects and **speech** to talk aloud.
> - **Robotics** to move and manipulate things.
>
> Together these six disciplines make up most of AI.

- **Successes:** in 2014 a chatbot called Eugene Goostman, pretending to be a 13-year-old boy, was reported to pass the test. Chatbots and voice assistants (Alexa, Siri, Cortana) are now everyday tools.
- **Limitations:** the test is not reproducible, not constructive and cannot be analysed mathematically. Most AI researchers spend little effort on passing it. It also asks whether we should copy humans at all.

> **Memory Hook: the flight metaphor**
>
> Aeroplanes did not succeed by flapping like birds. They succeeded by studying engines and aerodynamics. In the same way, AI should study the *principles* of intelligence, not just imitate humans.

Thinking rationally: the "laws of thought" approach

- Starts with Aristotle: what counts as a correct argument? Codify "right thinking" as irrefutable reasoning (syllogisms).
- Classic example: "All men are mortal. Socrates is a man. Therefore Socrates is mortal." This leads to **logic and logical inference**.
- The **logicist tradition** hopes to build intelligent systems from logic. Slogan: **algorithm = logic + control**. Examples are logic programming and automated deduction (propositional, first-order, modal, temporal logics).
- Two limits: it is hard to turn informal real-world knowledge into formal logic, and many logic problems are undecidable or NP-hard (computationally very expensive).

Acting rationally: the rational agent approach (the one this course follows)

An **agent** is something that perceives and acts, and keeps going over a long period of time. A **rational agent** acts to achieve the **best outcome**, or the **best expected outcome** when things are uncertain.

- It needs every skill from the Turing Test list, and sometimes logical thinking (for planning), and sometimes no thinking at all (a reflex like pulling your hand from a hot stove).
- **Advantage 1:** more general than "laws of thought", since correct inference is only one way to be rational.

- **Advantage 2:** easier to turn into real science than copying humans, because rationality is mathematically well defined.
- **Limited rationality:** perfect rationality is too expensive in complex worlds, so real agents must act sensibly even when they run out of time to compute everything.

Ways of classifying AI systems

- **Weak AI vs Strong AI.** Weak: can we build systems that *act as if* they are intelligent? Strong: can we build systems that *really are* intelligent (conscious, with wills and sentiments)?
- **General AI vs Narrow AI.** General: copes with any task, like a human. Narrow: handles one specific task (for example, only chess).
- **Symbolic (top-down) vs Connectionist (bottom-up).** Symbolic AI stores knowledge as logic, rules and ontologies, so humans can read the model. Connectionist AI uses neural networks, where knowledge is hidden in connection strengths, learned from examples, and humans cannot easily read it.

> **Memory Hook**
>
> Think Humanly / Act Humanly / Think Rationally / Act Rationally. Remember the pairs: Cognitive science, Turing, Logic, Agent. This course picks the last one: **the rational agent**.

1.4 The Fields That Built AI

AI is a team effort. Every field below handed it a tool.

- **Philosophy:** logic, reasoning, the mind as a physical system, the foundations of learning, language and rationality.
- **Mathematics:** formal proof, computation, what can and cannot be decided, tractability and probability.
- **Economics:** the formal theory of rational decisions and game theory.
- **Neuroscience:** the physical basis of mental activity.
- **Psychology:** adaptation, perception and motor control.
- **Computer Science and Engineering:** algorithms, data structures and efficient implementation.
- **Control Theory and Cybernetics:** stable (homeostatic) systems and optimal agent design.
- **Linguistics:** knowledge representation and grammar.

> **Memory Hook**
>
> PM-E-NPC-CL: Philosophy, Maths, Economics, Neuroscience, Psychology, Computer science, Control theory, Linguistics.

1.5 A Short History of AI

> **Watch First**
>
> Start with one of these, then come back and read the notes. Links to the Uninformed vs Informed and BFS lectures open the video directly. The others open a Gate Smashers search for that exact lecture title, so tap the top result.
>
> - Gate Smashers: History of Artificial Intelligence

Read this as a story of hype, disappointment and comeback.

- **1943 to 1955, the gestation.** 1943: McCulloch and Pitts built a model of artificial Boolean neurons. 1950: Turing published his famous paper and the Turing Test. 1951: Minsky and Edmonds built the first neural network computer.
- **1956, the birth of AI.** The Dartmouth Workshop gathered the top minds. Newell and Simon showed the **Logic Theorist**, the first non-numerical thinking program, which proved theorems from *Principia Mathematica*.
- **1952 to 1969, great expectations.** The **General Problem Solver** imitated human problem solving by attacking subgoals. John McCarthy invented **LISP** and time-sharing, and the logic-based Advice Taker. Minsky studied **microworlds** such as the blocks world. Early neural network work continued.
- **1966 to 1973, the collapse.** Progress was slower than promised. Systems did not scale because of **combinatorial explosion**. Minsky and Papert (1969) showed limits of neural networks.
- **Late 1960s to 1980s, knowledge-based systems.** Move from general-purpose to domain-specific. **DENDRAL** inferred molecular structure from mass-spectrometer data. **MYCIN** diagnosed blood infections using about 450 rules. In 1982 the expert system **R1 at DEC** saved about 40 million dollars a year. AI became an industry, then the expert system bubble burst: the **AI Winter**.
- **1986 onward, neural networks return.** The **back-propagation** algorithm was reinvented, reviving connectionist models.
- **1987 onward, AI adopts the scientific method.** Claims must rest on theorems or hard experimental evidence, not intuition. Probability and uncertainty took centre stage: **hidden Markov models** and **Bayesian networks**.
- **1995 onward, intelligent agents.** Focus on the "whole agent" acting in real environments with continuous sensor input (internet bots, robots, decision support).
- **2001 onward, big data.** Huge data sets and powerful hardware (GPUs) let deep networks train properly. Emphasis shifted from algorithms to data and learning instead of hand-coded knowledge.
- **2015 onward, the deep learning tsunami.** In 2016 **AlphaGo** beat the champion Lee Sedol at Go.

> **Memory Hook: the roller coaster**
>
> Boom (1956 to 1969), Bust (1966 to 1973), Boom (expert systems), Bust (AI Winter), then a steady climb: neural nets, probability, agents, big data, deep learning. **Hype cycles repeat.**

1.6 AI Today and Where It Is Published

AI is already inside daily life: search engines, route planning and traffic, logistics, medical and machine diagnosis, automated help desks, spam and fraud detection, smart cameras and homes, and product recommendations.

- Classify e-mail as spam, predict stock prices, read handwriting, learn to grasp a cup.
- Design molecules with given properties, translate Chinese to English, convert voice to text.
- Predict traffic trajectories, caption images automatically, drive autonomously.
- Run and jump on two legs, and beat a top pilot in a simulated F-16 dogfight.

Where AI research is published (just names to recognise)

- **Journals:** Artificial Intelligence; Journal of AI Research; IEEE Transactions on Pattern Analysis and Machine Intelligence; IEEE Intelligent Systems.
- **Conferences:** IJCAI (International Joint Conference on AI), AAAI, ECAI (European Conference on AI).
- **Societies:** AAAI, ACM SIGART, AISB.

PART 2

Intelligent Agents

The central idea of the course: an agent senses its world, thinks, and acts. Learn how to describe one, judge one and design one.

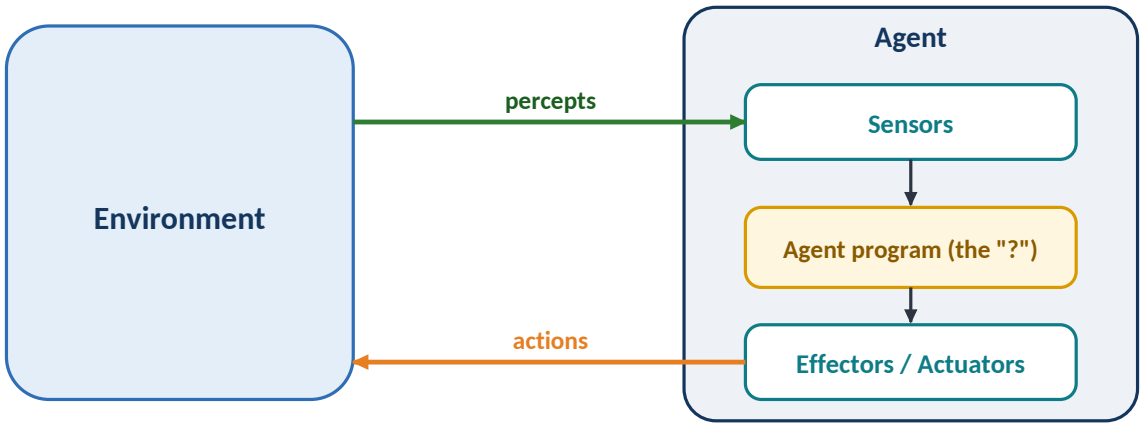
2.1 Agents and Environments

Watch First

Start with one of these, then come back and read the notes. Links to the Uninformed vs Informed and BFS lectures open the video directly. The others open a Gate Smashers search for that exact lecture title, so tap the top result.

- Gate Smashers: Intelligent Agents in Artificial Intelligence
- Gate Smashers: PEAS representation in AI

An **agent** is anything that can be viewed as **perceiving** its environment through **sensors** and **acting** upon that environment through **effectors** (also called actuators). Humans, robots, software bots (softbots) and even a thermostat all count.



The agent loop: percepts flow in through sensors, the agent program decides, actions flow out through effectors.

- The **agent function** maps from percept histories to actions: f : P* → A (P* means "a sequence of percepts").
- The **agent program** is the code that runs on the physical **architecture** to produce that function.
- **agent = architecture + program.**

Concept Box: Sensors, percepts, effectors, actions (using a human)

Take yourself as the agent. The slides split things into four layers:

- **Sensors:** eyes (vision), ears (hearing), skin (touch), tongue (taste), nose (smell), and the neuromuscular system (the sense of where your limbs are, called proprioception).
- **Percepts:** at the lowest level, raw electrical signals from those sensors. After preprocessing, they become objects in your visual field (location, texture, colour) and sound streams (pitch, loudness, direction).
- **Effectors:** limbs, fingers, eyes, tongue.
- **Actions:** lift a finger, turn left, walk, run, carry an object.

The key lesson: percepts and actions must be **carefully defined, possibly at different levels of abstraction**. "Electrical signal" and "I see a red car" are both percepts, but at different levels.

Example: the self-driving car

- **Percepts:** video, lidar, speedometer, odometer, engine sensors, keyboard input, microphone, GPS.
- **Actions:** steer, accelerate, brake, horn, speak or display.
- **Goals:** safety, reach the destination, maximise profit (fuel, tyre wear), obey laws, speed, passenger comfort.
- **Environment:** urban streets, freeways, traffic, pedestrians, weather, customers.
- Different aspects of driving may need different kinds of agent programs.

2.2 The Vacuum Cleaner World

Our favourite toy problem. There are two squares, **A** and **B**. Each can be clean or dirty, and the vacuum is in one of them.

- **Percepts:** location and contents, for example [A, Dirty].
- **Actions:** Left, Right, Suck, NoOp (do nothing).

Two ways to write the same agent

As a table (percept sequence to action): [A, Clean] → Right; [A, Dirty] → Suck; [B, Clean] → Left; [B, Dirty] → Suck; [A, Clean],[A, Clean] → Right; and so on forever.

```
function VACUUM-AGENT(location, status):
    if status = Dirty:   return Suck
    if location = A:     return Right
    if location = B:     return Left
```

Concept Box: Why the table has "dots" at the bottom, and why the small program wins

The table is indexed by the whole **percept sequence**, not just the latest percept. So [A, Clean] and [A, Clean],[A, Clean] are *different* rows, and so is every longer history. The table never ends, which is the meaning of the dots.

The three-line function behaves the same way using only the current percept. That answers the slide's two questions:

- **What is the right function?** The one that cleans dirt and does not waste moves. It depends on the performance measure (next section).
- **Can it be a small program?** Yes. A tiny rule-based program can replace an infinite table. This is the whole point of agent design.

2.3 Rationality: Doing the Right Thing

A rational agent is not one that always wins. It is one that makes the best decision it can with the information it has.

Concept Box: The performance measure: "right" needs a score

Before calling an action rational we need a **fixed performance measure** that scores the sequence of environment states. For the vacuum world, the slides suggest three different scores:

- One point per square cleaned up in time T.
- One point per clean square per time step, minus one point per move.
- A penalty if there are more than k dirty squares.

Why it matters: the first score rewards a vacuum that dirties and re-cleans. The second punishes wasteful movement. Change the score and the "best" agent changes. Always design the measure for what you really want, not what is easy to measure.

Other measures you may be asked about: false alarm (false positive) rate, false dismissal (false negative) rate, speed, resources used, and impact on the environment.

Concept Box: The definition of a rational agent

For every possible percept sequence, a rational agent chooses the action that **maximises the expected value of the performance measure**, given (1) the percept sequence so far and (2) its built-in and acquired knowledge.

Break it into three plain ideas: (a) it has a **score**, (b) it uses what it has **seen so far** and what it **already knows**, (c) it picks the action with the **best expected** score, not a guaranteed one.

Concept Box: Rational is not the same as omniscient, clairvoyant or successful

Not omniscient: percepts may not show everything relevant. **Not clairvoyant:** an action may not turn out as expected. **Therefore rational does not mean successful.**

Example: you cross a road at a green light after looking both ways. A speeding car runs the red light and hits you. You were rational (you used your information well) but not successful (bad luck). Rationality is judged by the *decision*, not by the lucky or unlucky *outcome*.

Concept Box: Rationality includes information gathering and learning

The slides say rationality is **not "rational ignorance"**: if you do not know something, **find out!** Agents can take actions just to change future percepts and gain information (exploration), such as looking both ways before crossing.

A rational agent therefore implies **exploration, learning and autonomy**. An agent is **autonomous** when its behaviour is determined by its **own experience** (it can learn and adapt). An agent that follows only what its designer wrote is **not** autonomous.

To survive, an agent needs enough built-in knowledge to start with **and** the ability to learn. Think of a newborn animal: born with reflexes, then learns the rest.

2.4 PEAS: Describing the Task Before You Build the Agent

Before designing an agent, always write down its task using **PEAS**: **P**erformance measure, **E**nvironment, **A**ctuators, **S**ensors.

Concept Box: PEAS worked out for four agents from the slides

Automated taxi driver

- Performance: safe, fast, legal, comfortable trip, maximise profit.
- Environment: roads, other traffic, pedestrians, customers.
- Actuators: steering wheel, accelerator, brake, signal, horn.
- Sensors: cameras, sonar, speedometer, GPS, odometer, engine sensors, keyboard.

Medical diagnosis system

- Performance: healthy patient, minimise costs and lawsuits.
- Environment: patient, hospital, staff.
- Actuators: screen display (questions, tests, diagnoses, treatments, referrals).
- Sensors: keyboard (symptoms, findings, patient's answers).

Part-picking robot

- Performance: percentage of parts placed in the correct bins.
- Environment: conveyor belt with parts, bins.
- Actuators: jointed arm and hand. Sensors: camera, joint angle sensors.

Interactive English tutor

- Performance: maximise the student's test score.
- Environment: a set of students.
- Actuators: screen display (exercises, suggestions, corrections). Sensors: keyboard.

Concept Box: Class exercises, with model answers

Internet book-shopping agent

- Performance: low price, good quality, books that match the request, fast and efficient.
- Environment: websites, vendors, shippers.
- Actuators: follow a link (URL), fill in forms, show results to the user.
- Sensors: web pages (text, graphics, scripts).
- Environment type: partially observable, stochastic, sequential, dynamic (prices change), discrete, multi-agent.

Robot soccer player

- Performance: goals scored, winning, avoiding fouls.
- Environment: field, ball, teammates, opponents, referee.
- Actuators: legs, kicking, turning, head. Sensors: camera, ball sensors, gyroscope, communication.
- Environment type: partially observable, stochastic, sequential, dynamic, continuous, multi-agent (cooperative with teammates, competitive with opponents).

Memory Hook

PEAS in a pod. Write the four letters down first. If you can list P, E, A and S, you have understood the problem.

2.5 Types of Environment

Watch First

Start with one of these, then come back and read the notes. Links to the Uninformed vs Informed and BFS lectures open the video directly. The others open a Gate Smashers search for that exact lecture title, so tap the top result.

- Gate Smashers: Types of environment in Artificial Intelligence

The environment largely decides what kind of agent you need. Six questions classify it.

Concept Box: 1. Fully observable vs partially observable (older slides call this accessible vs inaccessible)

Fully observable: the sensors give the complete state of the environment relevant to choosing an action at every moment. This is convenient because the agent need not keep track of the world itself.

Example: checkers is fully observable (you can see all pieces). Poker is partially observable (you cannot see the opponent's cards).

Concept Box: 2. Deterministic vs stochastic (nondeterministic)

Deterministic: the next state is completely determined by the current state and the agent's action. Moving a chess pawn from A2 to A3 always works. If an environment is deterministic except for the actions of other agents, it is called **strategic**.

If it is both accessible and deterministic, the agent does not have to deal with uncertainty. An environment that is not fully observable or not deterministic is called **uncertain**.

Concept Box: 3. Episodic vs sequential

Episodic: experience is split into atomic episodes (perceive, then do one action). Each choice depends only on the current episode.

Example: spotting defective parts on an assembly line. Each part is judged on its own. No planning ahead needed.

Sequential: the current decision can affect all future ones. Chess and taxi driving are sequential, because short-term actions have long-term consequences. Episodic environments are much simpler.

Concept Box: 4. Static vs dynamic (and semidynamic)

Static: the world does not change while the agent is thinking, so the agent need not keep watching the world or worry about time. Crossword puzzles are static.

Dynamic: the world keeps changing, and if the agent has not decided yet, that counts as deciding to do nothing. Taxi driving is dynamic.

Semidynamic: the world itself does not change with time, but the agent's score does. Chess played with a clock is semidynamic.

Concept Box: 5. Discrete vs continuous

Discrete: a limited number of clearly defined percepts and actions. Checkers has a finite number of board positions and moves.

Continuous: values sweep through a range. A patient's temperature and blood pressure are continuous. Taxi driving is a continuous-state, continuous-time problem (speeds, positions, steering angles).

The discrete versus continuous question applies to the state, to time, and to percepts and actions.

Concept Box: 6. Single agent vs multi-agent

Solving a crossword alone is **single-agent**. Playing chess is **multi-agent** (two agents). Multi-agent worlds can be **competitive** (chess) or **cooperative** (taxis avoiding each other).

The slides also list "with or without intelligent adversaries": if rational adversaries exist, the agent must think about strategy and game theory.

Memory Hook

Our Dog Eats Sausages, Dozing Alone: **O**bservable, **D**eterministic, **E**pisodic, **S**tatic, **D**iscrete, **A**gents (single or multi). Six questions, always asked in this order.

Classifying real examples (from the slides)

Using the older vocabulary: accessible / deterministic / episodic / static / discrete.

- **Image classification:** yes, yes, yes, yes, no (pixel values are continuous).
- **Solitaire:** no (hidden cards), yes, no, yes, yes.
- **Backgammon:** yes, no (dice), no, yes, yes.
- **Customer support chat:** no, no, no, no, yes.
- **Taxi driving:** no, no, no, no, no. The hardest case.

And with the newer vocabulary (fully observable, deterministic, episodic, static, discrete, single agent):

- **Chess with a clock:** yes, strategic, no, semi, yes, no.
- **Chess without a clock:** yes, strategic, no, yes, yes, no.
- **Taxi driving:** no, no, no, no, no, no.

Memory Hook

The real world is partially observable, stochastic, sequential, dynamic, continuous and multi-agent. That is the hardest setting of all six questions, and the one AI is ultimately trying to handle.

2.6 Agent Functions and Agent Programs

An agent is completely specified by its **agent function** (percept sequence to action). One agent function, or a small family of equivalent ones, is rational. The goal is to implement that rational function **concisely**.

Concept Box: The table-driven (table-lookup) agent

The simplest design: keep a huge table of percept sequence to action, and just look up the answer.

Why it fails (the slides list four problems):

- The table is **too big** to generate or store. Chess alone has about 10^{120} states.
- It takes forever to **build** the table, and there is **no autonomy**; even with learning it takes a long time to learn every entry.
- It has **no knowledge of non-perceptual parts** of the state.
- It is **not adaptive**: if the environment changes, the whole table must be rewritten. It also cannot make actions depend on past states (it can loop).

2.7 The Five Kinds of Agent

Watch First

Start with one of these, then come back and read the notes. Links to the Uninformed vs Informed and BFS lectures open the video directly. The others open a Gate Smashers search for that exact lecture title, so tap the top result.

- Gate Smashers: Model Based Reflex Agent with Real Life Examples
- Gate Smashers: Goal Based Agents in AI
- Gate Smashers: Simple Reflex and Utility Based Agents

The slides list five basic types, in order of increasing generality: **simple reflex, model-based reflex (agents with memory), goal-based, utility-based and learning agents.** The first deck also lists the table-driven agent as a starting point.

Concept Box: Simple reflex agents: "if this, then that"

They use **condition-action rules** (a production system) to map the current percept to an action. They are **stateless**: they have no memory of past world states.

Example: a thermostat or the vacuum agent above: "if dirty then suck".

Problems: the rule set is usually still too big; they cannot see non-perceptual parts of the state; they are not adaptive; and they cannot make actions depend on previous states.

Concept Box: Agents with memory (model-based reflex agents)

They keep an **internal state** that remembers past percepts. This is needed because sensors rarely show the whole world at once, so the picture of the world is **built up over time**. The state also tells apart **different world states that produce the same percept**.

Example: a driving agent behind a lorry cannot see a car in the next lane now, but remembers it was there a second ago. The extra boxes on the diagram are "how the world evolves" and "what my actions do".

Example from the slides: Rodney Brooks' **Subsumption Architecture**, described in the next box.

Concept Box: Brooks' Subsumption Architecture

Main idea: build complex robots by **decomposing behaviour into a hierarchy of skills**. Each skill is a complete percept-action cycle for one specific task, such as avoiding contact, wandering, exploring or recognising doorways.

Behaviours are **loosely coupled and asynchronous**. Higher layers can override (subsume) lower ones, like a reflex that stops you walking into a wall even while you are daydreaming about where to go.

Concept Box: Goal-based agents: thinking about the future

They add **goal information**, a description of desirable situations, to the state. They choose actions that achieve a given or computed goal and consider **future events**: "what will happen if I do this?"

They are **deliberative** instead of reactive, and may consider long sequences of actions. Keeping track of the current state alone is often not enough, because you need a goal to decide which situations are good.

Example: a route planner choosing a path to Lahore. Search and planning are the tools such agents use, which is exactly what the next parts of these notes cover.

Concept Box: Utility-based agents: not just "goal reached", but "how happy"

A goal is a yes or no (happy or unhappy). When many alternatives exist, a **utility function** gives a score:

U : State → Real numbers, a measure of success or happiness in a given state.

It lets the agent compare **conflicting goals** (fast vs safe) and weigh the **likelihood of success against the importance of the goal** when success is uncertain. Utility-based agents base decisions on classic utility theory.

Example: two routes both reach the goal. One is shorter but through heavy traffic, the other is longer but calmer. Utility turns "comfort" and "time" into one number so the agent can pick.

Concept Box: Learning agents

A learning agent improves with experience. It has a part that acts and a part that learns from feedback so it can adapt to new situations. This ties back to autonomy: the more an agent learns from its own experience, the less it depends on the designer.

Memory Hook

Really Mighty Goats Use Legs: **R**eflex, **M**odel (memory), **G**oal, **U**tility, **L**earning. Each one adds something to the last: memory, then a goal, then a score, then the ability to improve.

2.8 Properties of Agents and a Quick Summary

Concept Box: A discrete agent

A **discrete** agent receives percepts **one at a time** and maps that sequence of percepts to a sequence of **discrete actions**. Think of a turn-based loop: percept in, action out, repeat.

- Other possible properties: **autonomous**, **reactive** to the environment, **pro-active** (goal-directed), and **interacts with other agents** through the environment.
- The slides define an **intelligent agent** as one that perceives its environment via sensors and acts **rationally** upon it with its effectors.
- An **ideal agent** always picks the action that maximises expected performance given its percept sequence so far.
- An **agent program** maps percept to action and updates its internal state: reflex agents respond immediately, goal-based agents act to achieve goals, utility-based agents maximise their own utility function.
- Representing knowledge well is important for good agent design.
- The most challenging environments are inaccessible, nondeterministic, nonepisodic, dynamic, continuous and contain intelligent adversaries.

PART 3

Uninformed Search: Finding a Path With No Hints

Searching a problem when all you know is the problem definition. Six classic strategies and how to judge them.

3.1 The Big Idea of Searching

Watch First

Start with one of these, then come back and read the notes. Links to the Uninformed vs Informed and BFS lectures open the video directly. The others open a Gate Smashers search for that exact lecture title, so tap the top result.

- Gate Smashers: Lec-5: Uninformed Vs Informed Search with Example
- Gate Smashers: Breadth First Search with example | Uninformed Search
- Gate Smashers: Depth First Search (DFS) with example | Uninformed Search
- Gate Smashers: Bidirectional Search Algorithm with Real Life Examples

A search problem is usually drawn as a **graph of states**. You start at the current state, then step by step explore paths until you reach a goal state. The only thing that differs between algorithms is **which node you expand next**. That choice is called the **search strategy**.

Concept Box: Four questions to judge any search strategy

Completeness: if a solution exists, is the algorithm guaranteed to find one in finite time?

Optimality: when it finds a solution, is it the best (least-cost) one?

Time complexity: how many nodes does it generate (worst case)?

Space complexity: how many nodes must it keep in memory at once (worst case)?

Time and space are written using three letters:

- **b** = maximum branching factor (how many children a node can have).
- **d** = depth of the least-cost (shallowest) solution.
- **m** = maximum depth of the state space, which may be infinite.

Picture it: if every node has 3 children (b = 3) and the goal is 4 levels down (d = 4), then there are 1 + 3 + 9 + 27 + 81 = 121 nodes down to depth 4. Increasing b or d makes this explode, which is why we care about complexity.

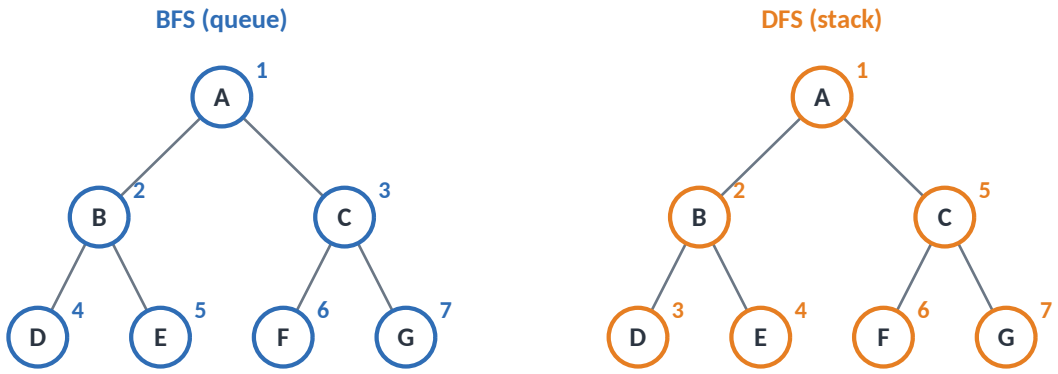
Uninformed (blind) search uses only the information in the problem definition and explores the graph without guidance. The six strategies are **BFS, UCS, DFS, DLS, Iterative Deepening and Bidirectional search**. You can also search **forward** (start to goal), **backward** (goal to start) or **bidirectionally** (both at once).

Memory Hook

Blind search = no map, no compass. Informed search (Part 4) = you get a compass pointing roughly to the goal.

3.2 Breadth-First Search (BFS)

BFS expands the **shallowest unexpanded node** first. Visit every node on the current level, then move to the next level. It uses a **FIFO queue** (first in, first out).



The same tree visited two ways. The small numbers show the visiting order. BFS: A B C D E F G. DFS: A B D E C F G.

Concept Box: BFS properties and what they mean

Complete: yes, if b is finite.

Optimal: yes if every step costs the same (for example 1). Otherwise not in general. BFS finds the solution with the **fewest steps**, not the lowest total cost.

Time: about $1 + b + b^2 + \ldots + b^d$ nodes, so exponential in d. The slides write $O(b^{d+1})$ because they count nodes generated while expanding the last level.

Space: the same order as time, because it **keeps every node in memory**. This is the big problem. The slide says a program producing 10 MB per second would produce 860 MB in 24 hours; the arithmetic really gives roughly 860 **GB**. Either way, memory runs out long before time does.

- **BFS is a good choice when:** memory is not a problem, you need the solution with the fewest arcs (steps), at least some solutions are shallow, or there may be infinite paths.
- **BFS is a bad choice when:** memory is limited, all solutions are deep, or the branching factor is huge.

3.3 Uniform-Cost Search (UCS)

UCS is BFS that cares about **cost**. It expands the **least-cost unexpanded node**, using a priority queue ordered by path cost g(n). If all step costs are equal, UCS behaves exactly like BFS.

Concept Box: A UCS example you can redo on paper

Graph: S to A costs 5, S to B costs 2, A to G costs 5, B to D costs 2, D to G costs 3. (This is the same graph used later in Part 4.)

- Start: frontier = {S (0)}. Expand S. Frontier = {B (2), A (5)}.
- Cheapest is B (2). Expand B. Add D (2+2 = 4). Frontier = {D (4), A (5)}.
- Cheapest is D (4). Expand D. Add G (4+3 = 7). Frontier = {A (5), G (7)}.
- Cheapest is A (5). Expand A. Add G (5+5 = 10). Frontier = {G (7), G (10)}.
- Cheapest is G (7). It is a goal, so stop. **Path S, B, D, G, cost 7.**

The goal is tested **when a node is chosen for expansion**, not when it is first generated. That is why UCS did not stop when G first appeared with cost 10.

- **Complete:** yes, provided every step cost is at least some small positive number (written step cost ≥ ε).

- **Optimal:** yes, because nodes are expanded in increasing order of cost.
- **Time and space:** $O(b^{ceil(C^*/\varepsilon)})$, where C^* is the cost of the optimal solution. It cannot be stated in terms of d alone, since cheap steps can make the path long.

3.4 Depth-First Search (DFS)

DFS expands the **deepest unexpanded node** first. It dives down one path as far as it can, and when it hits a dead end it **backtracks** to the most recent node with unexplored children. It uses a **stack** (LIFO, last in first out), or recursion.

> **Concept Box: DFS properties and what they mean**
>
> **Complete:** no in infinite-depth spaces or spaces with loops (it can dive forever). It is complete in finite spaces, and you can fix loops by avoiding repeated states along a path.
>
> **Optimal:** no. It returns the first solution it stumbles on, which may be deep and expensive.
>
> **Time:** $O(b^m)$. Bad if m is much bigger than d. But if solutions are dense, DFS can beat BFS.
>
> **Space:** O(bm), which is **linear**. Only the current path plus its unexpanded siblings are stored. This is DFS's superpower.

- **DFS is a good choice when:** memory is tight (for example, robots with complex state), or there are many solutions, possibly long, such as when every path leads to a solution.
- **DFS is a bad choice when:** the space has infinite branches or cycles, or shallow solutions exist.

> **Memory Hook: BFS vs DFS**
>
> BFS = Queue, spreads Broad, finds the Fewest-steps answer, eats memory. DFS = Stack, goes Deep, uses little memory, can get lost forever.

3.5 Depth-Limited Search (DLS)

To fix DFS's "dive forever" problem, set a **depth limit l** and refuse to expand nodes at depth l. Nodes at depth l are treated as if they have no children. It is the same as DFS when l = infinity. DLS can end in two ways: **failure** (the whole limited tree has no solution) or **cutoff** (the limit was hit, so a solution might exist deeper).

> **Concept Box: DLS properties, with a correction**
>
> **Time:** $O(b^l)$. **Space:** O(bl). **Optimal:** no.
>
> **Complete:** only if the solution is within the limit, that is **d ≤ l**. Your slide prints "Yes if l < d", which looks like a typo, because a limit shallower than the solution cannot find it. Use d ≤ l in your own reasoning. If your teacher marks strictly by the slide, quote the slide in the exam, but know the logic.
>
> **Example:** a goal at depth 5 with limit l = 3 gives cutoff, with no answer. Raise the limit to 5 or more and it works.

3.6 Iterative Deepening Search (IDS)

What if you do not know the right limit? **Try l = 0, then 1, then 2, 3, ... until a goal is found.** Each round is a depth-limited DFS. You get BFS's completeness with DFS's tiny memory.

Concept Box: Does repeating work make IDS slow? A numerical check

Suppose b = 10 and the goal is at depth d = 5.

- BFS (or one DFS to depth 5) generates 1 + 10 + 100 + 1,000 + 10,000 + 100,000 = **111,111** nodes.
- IDS runs depth limits 0 to 5, generating 1 + 11 + 111 + 1,111 + 11,111 + 111,111 = **123,456** nodes in total.

The extra work is only about **11 percent**, because almost all nodes live at the deepest level and the repeated upper levels are cheap. So IDS pays a tiny time cost to get BFS-like behaviour with DFS-like memory.

- **Complete:** yes. **Time:** $O(b^d)$. **Space:** O(bd). **Optimal:** yes if step cost is 1 (and it can be modified to follow uniform cost).

Memory Hook

IDS = DFS memory + BFS answer. When you must pick an uninformed method for a huge tree with unknown depth, IDS is the usual winner.

3.7 Avoiding Repeated States

Wasting time on states you have already seen can turn a **linear** problem into an **exponential** one. Repeated states are unavoidable when actions are reversible, such as route finding (you can drive back) and sliding-block puzzles (you can slide a tile back). The fix is to remember the states you have already expanded (an "explored" list) and skip them. This turns tree search into **graph search**.

3.8 Bidirectional Search

Search **forward from the start and backward from the goal at the same time**, and stop when the two searches **meet in the middle**. Each side only needs to go about half the depth, so time drops from b^d to about $2 \times b^{d/2}$.

Concept Box: The catch: what does "search backwards from the goal" mean?

You need a way to generate the **predecessors** of the goal, and that can be difficult:

- What are the predecessors of "checkmate" in chess?
- What if there are **several** goal states?
- What if there is only a goal *test* and no explicit list of goals?

You also need to keep track of where the two sets of explored nodes intersect.

3.9 Uninformed Search at a Glance

Cheat lines to memorise

- **BFS:** complete (finite b), optimal if equal costs, time and space exponential, queue.
- **UCS:** complete (step cost ≥ ε), optimal, expands by lowest g(n), priority queue.
- **DFS:** not complete, not optimal, time $O(b^m)$, space O(bm) linear, stack.
- **DLS:** complete if d ≤ l, not optimal, time $O(b^l)$, space O(bl).
- **IDS:** complete, optimal if step cost 1, time $O(b^d)$, space O(bd).
- **Bidirectional:** about $O(b^{d/2})$ time, but needs predecessors of the goal.

PART 4

Informed Search: Search With a Compass

Using problem-specific knowledge (a heuristic) to find solutions faster. Greedy best-first search, A*, memory-bounded RBFS and how to design good heuristics.

4.1 Best-First Search and the Heuristic Function

Watch First

Start with one of these, then come back and read the notes. Links to the Uninformed vs Informed and BFS lectures open the video directly. The others open a Gate Smashers search for that exact lecture title, so tap the top result.

- Gate Smashers: Uninformed Vs Informed Search with Example
- Gate Smashers: Best First Search Algorithm, How it Works, All Imp Points

An **informed (heuristic) search** uses knowledge about the problem **beyond its definition**, so it can find solutions more efficiently than an uninformed one.

Concept Box: Best-first search: one skeleton, many algorithms

Best-first search picks the next node to expand using an **evaluation function f(n)**, thought of as a cost estimate. The node with the **lowest f** is expanded first.

Its code is the same as uniform-cost search, except the priority queue is ordered by **f(n)** instead of the path cost g(n). So the **choice of f decides the algorithm**:

- f(n) = g(n) gives uniform-cost search.
- f(n) = h(n) gives **greedy best-first search**.
- f(n) = g(n) + h(n) gives **A***.

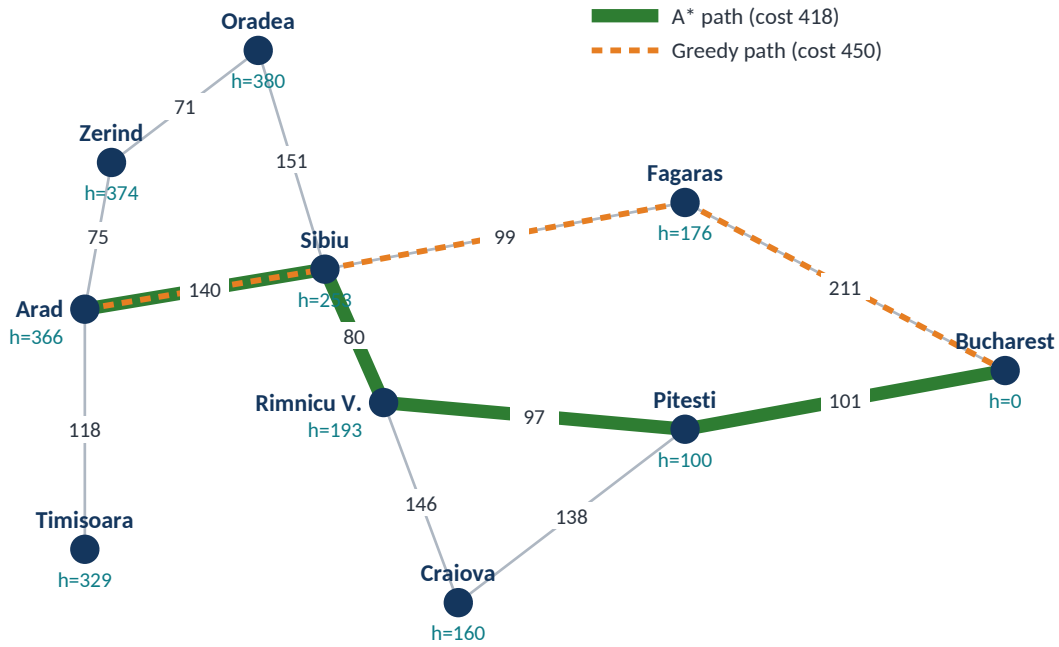
Concept Box: What exactly is a heuristic h(n)?

h(n) = the estimated cost of the cheapest path from the state at node n to a goal state. It is a guess about the future, and it is the most common form of extra knowledge an informed search gets.

Rules: h is **non-negative**, depends on the problem, and **h(goal) = 0** (at the goal there is nothing left to pay).

Example (Romania map): if the goal is Bucharest, use $h_{SLD}(n)$ = the **straight-line distance** from city n to Bucharest. You cannot compute it from the road map itself; you need extra information (a table of straight-line distances). It works well because straight-line distance is correlated with real road distance.

Everyday analogy: when you are lost in a city and can see the tall tower you are heading to, "distance to the tower" is your heuristic. It does not tell you the roads, but it points you the right way.



A simplified Romania map from the slides. Edge numbers are road distances in km. The h value under each city is the straight-line distance to Bucharest.

4.2 Greedy Best-First Search (GBFS)

Watch First

Start with one of these, then come back and read the notes. Links to the Uninformed vs Informed and BFS lectures open the video directly. The others open a Gate Smashers search for that exact lecture title, so tap the top result.

- Gate Smashers: Best First Search Algorithm in Artificial Intelligence

GBFS expands the node that **appears closest to the goal**, hoping to reach it quickly. It ignores everything about the road already travelled and looks only at the heuristic:

$$f(n) = h(n)$$

Concept Box: GBFS on the Romania map, step by step (Arad to Bucharest)

The numbers are h_{SLD} values. The frontier is always sorted by smallest h.

- **Start:** frontier = {Arad 366}. Expand Arad.
- **After Arad:** frontier = {Sibiu 253, Timisoara 329, Zerind 374}. Smallest is **Sibiu**, so expand it.
- **After Sibiu:** frontier = {Fagaras 176, Rimnicu Vilcea 193, Timisoara 329, Zerind 374, Oradea 380}. Smallest is **Fagaras**.
- **After Fagaras:** frontier = {Bucharest 0, Rimnicu Vilcea 193, ...}. Bucharest has h = 0, so it is chosen and the goal is reached.

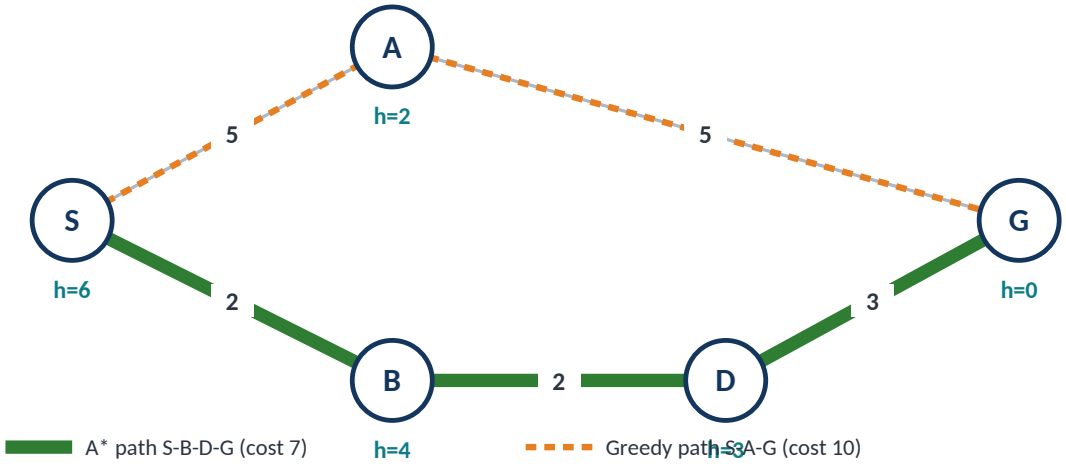
Result: Arad, Sibiu, Fagaras, Bucharest = 140 + 99 + 211 = **450 km**. It never expanded a useless city, so the search cost was tiny. But the best route is 418 km (through Rimnicu Vilcea and Pitesti), so the answer was **32 km longer than optimal**.

Concept Box: Why "greedy" and why it is not optimal

It is greedy because it always grabs the option that looks best **right now**, like choosing the restaurant that looks closest on the map without checking whether the road to it is closed or a long detour. It never looks at **g(n)**, the cost already spent, so it cannot notice when an apparently close city is actually reached by an expensive road.

Small example where greedy loses. Graph: S to A cost 5, S to B cost 2, A to G cost 5, B to D cost 2, D to G cost 3. Heuristics: h(S)=6, h(A)=2, h(B)=4, h(D)=3, h(G)=0.

- Greedy at S compares h(A)=2 with h(B)=4, picks A, then goes to G. Path S, A, G costs **10**.
- But S, B, D, G costs only **7**. Greedy missed it because B looked farther away.



The trap in one picture. Greedy follows the lowest h and pays 10. A* weighs g and h together and finds the cost-7 path.

Concept Box: GBFS properties

Complete? No. It can get stuck in loops if it does not check repeated states. It is complete in a finite space **with** repeated-state checking.

Time: $O(b^m)$ in the worst case, where m is the maximum depth. A good heuristic gives a dramatic improvement.

Space: $O(b^m)$, because it keeps all nodes in memory.

Optimal? No (shown above).

Memory Hook

GBFS looks only at the **finish line** (h). It forgets how far it has already run (g). Fast, but it can be led astray.

4.3 A* Search

Watch First

Start with one of these, then come back and read the notes. Links to the Uninformed vs Informed and BFS lectures open the video directly. The others open a Gate Smashers search for that exact lecture title, so tap the top result.

- Gate Smashers: A* algorithm in AI with example
- Gate Smashers: How to Prove A* Admissible | Underestimation and Overestimation

A* fixes greedy's blindness to cost. It **minimises the total estimated solution cost** by adding what you have paid so far to what you think is left:

$$f(n) = g(n) + h(n)$$

- **g(n)** = actual cost from the start to node n (the cost so far).
- **h(n)** = estimated cost from n to the goal.
- **f(n)** = estimated cost of the cheapest solution that passes through n.
- The idea is to **avoid expanding paths that are already expensive**.

Concept Box: A* on the Romania map, step by step

Each frontier value is f = g + h. For example, Sibiu: g = 140 (Arad to Sibiu) and h = 253, so f = 393.

- **Start:** frontier = {Arad 366}. Expand Arad.
- **After Arad:** Sibiu 393 (140+253), Timisoara 447 (118+329), Zerind 449 (75+374). Lowest is Sibiu, expand it.
- **After Sibiu:** Rimnicu Vilcea 413 (220+193), Fagaras 415 (239+176), Timisoara 447, Zerind 449, Oradea 671 (291+380). Lowest is Rimnicu Vilcea.
- **After Rimnicu Vilcea:** Fagaras 415, Pitesti 417 (317+100), Timisoara 447, Zerind 449, Craiova 526 (366+160), Oradea 671. Lowest is Fagaras.
- **After Fagaras:** Pitesti 417, Timisoara 447, Zerind 449, **Bucharest 450** (239+211+0), Craiova 526, Oradea 671. Lowest is Pitesti.
- **After Pitesti:** a cheaper route to Bucharest appears: **418** (317+101). It replaces the 450 entry. Now Bucharest 418 is the lowest, it is chosen, and it is the goal.

Result: Arad, Sibiu, Rimnicu Vilcea, Pitesti, Bucharest = 140 + 80 + 97 + 101 = **418 km, the optimum.**

Spot the difference with greedy: A* did not stop when Bucharest first showed up with 450. It waited until the cheaper 418 route was available, because a goal is only accepted when it is **chosen** for expansion.

Concept Box: Admissible heuristics (the first condition for optimality)

A heuristic is **admissible** if it **never overestimates** the true cost to the goal. Formally, for every node n: **$h(n) \leq C(n)$**, where C(n) is the real cheapest cost from n to the goal. It is "optimistic".

Example: straight-line distance is admissible, because the shortest path between two points is a straight line, so no road can be shorter than it.

Why it helps: if h is admissible, f(n) = g(n) + h(n) never overestimates the cost of a solution through n, because g(n) is exact and h(n) is optimistic.

Warning example: if a city is truly 100 km from the goal and your heuristic says 150, it overestimates. A* might wrongly avoid a good path.

Concept Box: Consistent heuristics (the second condition)

A heuristic is **consistent** if for every node n and every successor n' reached by action a:

$h(n) \leq c(n, a, n') + h(n')$

In words: the estimate from n is never more than the step cost to the next node plus the estimate from there. This is the **triangle inequality** (going directly is never longer than going around). Straight-line distance is consistent.

Relationship: every consistent heuristic is admissible, but not every admissible one is consistent. **If h is admissible and consistent, A* is complete and optimal.**

Concept Box: Why A* is optimal (the proof in plain words)

Step 1. With a consistent h, the f-values never decrease along a path. Proof: if n' is a successor of n, then g(n') = g(n) + c(n, a, n'), so f(n') = g(n) + c(n, a, n') + h(n') ≥ g(n) + h(n) = f(n), using consistency.

Step 2. Whenever A* picks a node n to expand, it has already found the best path to n. (If not, some other frontier node on the best path would have a smaller f and would have been picked first.)

Step 3. So nodes are expanded in **non-decreasing f order**. At a goal, h = 0, so f is the true cost. The first goal chosen therefore has the lowest cost, and any later goal costs at least as much.

Picture: A* grows **f-contours**, like ripples in a pond. It finishes all nodes with $f = f_i$ before touching f_{i+1}.

Concept Box: A* properties

Complete: yes, unless there are infinitely many nodes with f ≤ f(goal).

Optimal: yes, since it cannot expand contour i+1 until contour i is finished.

Time: exponential in the worst case, but strongly depends on the quality of h.

Space: $O(b^m)$; it keeps all generated nodes in memory. This is A*'s main weakness.

A* expands **all** nodes with f(n) < C*, **some** nodes with f(n) = C*, and **none** with f(n) > C*, where C* is the optimal cost. Among algorithms that use the same heuristic, A* is **optimally efficient**: no other optimal algorithm expands fewer nodes.

Memory Hook: three formulas

Greedy: f = h (hope). UCS: f = g (history). **A*: f = g + h (history plus hope).** The best of both, provided h never lies by overestimating.

4.4 Recursive Best-First Search (RBFS), the Memory-Saving Version

A* is great but remembers everything. **RBFS mimics best-first search while using only linear space.** Its structure looks like recursive DFS, but instead of diving forever it carries an **f_limit**: the f-value of the **best alternative path** available from any ancestor.

Concept Box: How RBFS works, in four ideas

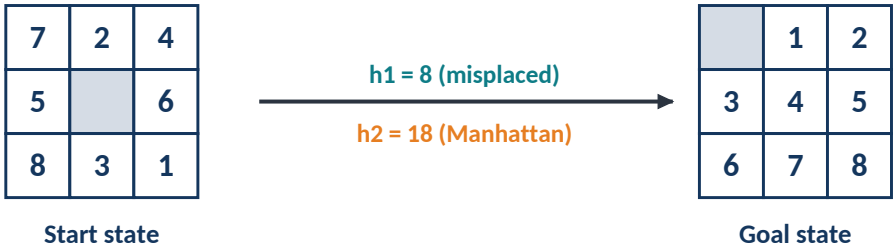
- Go down the most promising path, just like DFS.
- If the current node's f exceeds f_limit (some other branch now looks better), **unwind** back to that alternative.
- As it unwinds, it **replaces the f-value of each node on the path with a backed-up value** (the best f-value of its children). This is a bookmark: it remembers how good the abandoned subtree was.
- If the abandoned subtree later looks best again, it can be **re-expanded** (some work is repeated, the price for saving memory).

Stages on the Romania example: expand Arad, then Sibiu, then Rimnicu Vilcea. Its f exceeds the limit set by Fagaras, so unwind to Sibiu. Expand Fagaras, which also turns out worse than the bookmark, so unwind again. Re-expand Rimnicu Vilcea, then Pitesti, and finally Bucharest is reached.

4.5 Designing Good Heuristics (the 8-Puzzle)

The **8-puzzle** is a 3x3 board with 8 tiles and one blank. You slide a tile into the blank to reach the goal arrangement. It was one of the earliest heuristic search problems.

- Average solution length for a random instance: about **22 steps**.
- Branching factor about **3**: the blank in the middle has 4 moves, in a corner 2 moves, along an edge 3 moves.
- A blind search might look at roughly **170,000 states**, since 9!/2 = 181,440 states are reachable. The 15-puzzle has about $\mathbf{10^{13}}$ states.
- So we need a good heuristic. For A* to give shortest solutions it must **never overestimate** the number of steps left.



The 8-puzzle example from the slides (shortest solution is 26 steps). Both heuristics give numbers below 26, so both are admissible.

Concept Box: Two admissible heuristics for the 8-puzzle

h_1 = number of misplaced tiles. Admissible because every misplaced tile needs at least one move. For the slide's start state h_1 = **8**.

h_2 = total Manhattan distance: for each tile, the horizontal plus vertical distance to its goal square, added up. Admissible because a move shifts one tile by one square, and tiles cannot move diagonally. For the slide's start state h_2 = 3+1+2+2+2+3+3+2 = **18**.

Both stay below the true cost of 26. Part 5 teaches how to calculate Manhattan distance by hand.

Concept Box: Measuring heuristic quality: the effective branching factor b*

If A* generates **N** nodes for a solution at depth **d**, then **b*** is the branching factor a perfectly uniform tree of depth d would need to contain **N + 1** nodes:

$$\mathbf{N + 1 = 1 + b^* + (b^*)^2 + \ldots + (b^*)^d}$$

Example from the slides: A* finds a solution at depth 5 using 52 nodes, so b* ≈ **1.92**. A very good heuristic has b* close to 1 (almost no wasted branching).

Experiment: 1200 random 8-puzzles (100 for each solution length 2, 4, ..., 24) were solved with iterative deepening and with A* using h_1 and h_2. In the textbook version of this experiment, at solution depth 12 the node counts are about 3.6 million (IDS), 227 (A* with h_1) and 73 (A* with h_2); at depth 24 they are about 39,000 for h_1 and 1,600 for h_2. **h_2 beats h_1, and both crush blind search.**

Concept Box: Dominance: bigger (but still admissible) is better

If **$h_2(n) \geq h_1(n)$ for every n** and both are admissible, then **h_2 dominates h_1**. A* with h_2 never expands more nodes than A* with h_1. Domination translates directly into efficiency: use the heuristic with higher values, as long as it is admissible (and consistent).

Combining heuristics: given admissible h_a and h_b, take **$h(n) = \max(h_a(n), h_b(n))$**. It is also admissible and dominates both.

Check with the example: Manhattan distance (18) is at least the misplaced-tile count (8), so h_2 dominates h_1.

Concept Box: Inventing heuristics: relaxed problems

A **relaxed problem** is the same problem with fewer restrictions on the actions. The exact solution cost of a relaxed version is an admissible heuristic for the real problem.

- Relax the 8-puzzle so a tile can move **anywhere**: the exact cost is the number of misplaced tiles = $\mathbf{h_1}$.
- Relax it so a tile can move to **any adjacent square** (even an occupied one): the exact cost is the Manhattan distance = $\mathbf{h_2}$.

Key point: the optimal cost of a relaxed problem is never greater than the optimal cost of the real problem, because the real solution is also a solution of the relaxed one. That is exactly why the heuristic never overestimates.

4.6 Informed Search Summary

Cheat lines to memorise

- Heuristic functions estimate the cost of the cheapest path to the goal; good ones slash search cost.
- **Greedy best-first:** expands lowest h. Incomplete and not always optimal.
- **A*:** expands lowest g + h. Complete and optimal (with admissible, consistent h), and optimally efficient.
- **RBFS:** A*-like behaviour in linear memory, at the price of re-expanding nodes.
- Admissible heuristics can be derived from the exact solution of **relaxed problems**.

PART 5

Special Lesson: Greedy Best-First Search with Manhattan Distance

A slow, step-by-step lesson. First Manhattan distance on its own, then GBFS on an 8-puzzle, then GBFS on a grid with walls, then real code.

5.1 Manhattan Distance From Scratch

Watch First

Start with one of these, then come back and read the notes. Links to the Uninformed vs Informed and BFS lectures open the video directly. The others open a Gate Smashers search for that exact lecture title, so tap the top result.

- Gate Smashers: 8 puzzle problem in Artificial Intelligence
- Gate Smashers: Best First Search Algorithm in Artificial Intelligence

Manhattan distance is the distance you would walk in a city laid out in blocks, where you can only go **horizontally or vertically** and never cut diagonally through buildings. The name comes from the street grid of Manhattan.

Concept Box: The formula and a quick example

For two cells A = (r_1, c_1) and B = (r_2, c_2) (row, column):

Manhattan distance = $|r_1 - r_2| + |c_1 - c_2|$

In words: how many rows apart, plus how many columns apart.

Example: from (1, 2) to (4, 6). Rows apart: $|1 - 4| = 3$. Columns apart: $|2 - 6| = 4$. Manhattan distance = 3 + 4 = **7**.

Compare with straight-line (Euclidean) distance for 3 rows and 4 columns apart: the straight line is 5 (the 3-4-5 triangle), but you must walk 3 + 4 = **7** blocks. Manhattan is always at least as large as the straight line.

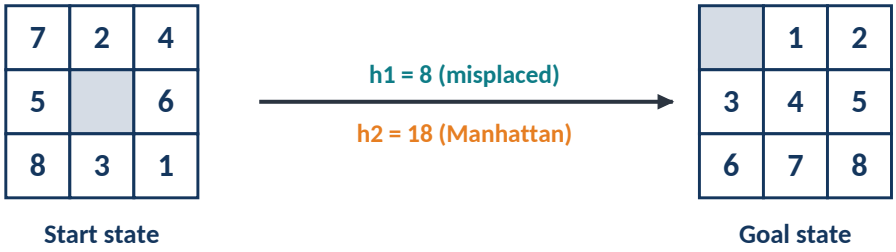
Concept Box: Why Manhattan distance is a legal (admissible) heuristic here

When every move goes one step up, down, left or right and costs 1, you need **at least** as many moves as the Manhattan distance, because each move reduces the row gap or the column gap by at most 1. So it **never overestimates**, which makes it admissible. It is also consistent.

When it stops being admissible: if diagonal moves are allowed (use Chebyshev or octile distance instead), or if some steps cost less than 1.

5.2 Manhattan Distance on the 8-Puzzle

For the 8-puzzle, add up the Manhattan distance of **every tile** (not the blank) from where it is now to where it belongs. Number rows and columns from 0, top-left.



The slides' example. Start has the blank in the centre. The goal has the blank in the top-left corner.

Concept Box: The slide example, tile by tile (answer 18)

Format: tile: current (row, col) to goal (row, col): |row gap| + |col gap|.

- **Tile 1:** (2,2) to (0,1): 2 + 1 = **3**
- **Tile 2:** (0,1) to (0,2): 0 + 1 = **1**
- **Tile 3:** (2,1) to (1,0): 1 + 1 = **2**
- **Tile 4:** (0,2) to (1,1): 1 + 1 = **2**
- **Tile 5:** (1,0) to (1,2): 0 + 2 = **2**
- **Tile 6:** (1,2) to (2,0): 1 + 2 = **3**
- **Tile 7:** (0,0) to (2,1): 2 + 1 = **3**
- **Tile 8:** (2,0) to (2,2): 0 + 2 = **2**

Total h_2 = 3 + 1 + 2 + 2 + 2 + 3 + 3 + 2 = 18. For comparison, misplaced tiles gives h_1 = 8. The real shortest solution is 26 moves, so both stay below it (admissible), and 18 is much closer to 26 than 8 is. That is why Manhattan is the better heuristic.

Do not count the blank. If you counted it you could overestimate, because moving the blank is not a separate job; it moves as a side effect of moving tiles.

5.3 The GBFS Recipe

Concept Box: Greedy best-first search in five steps

- Put the start node in a **priority queue**, with priority = h(n) (here, the Manhattan distance to the goal).
- **Pop** the node with the **smallest h**.
- If it is the **goal**, stop and read back the path.
- Otherwise create its neighbours. **Skip any state already seen.** Compute each neighbour's h with the Manhattan formula and push them into the queue.
- Go back to step 2.

That is it. The only difference from A* is the priority: GBFS uses **h alone**, A* uses **g + h**.

Memory Hook

GBFS = "closest-looking first". Calculate h for every neighbour, step toward the smallest one, never look at how far you have already walked.

5.4 Worked Example A: GBFS on an 8-Puzzle

This time the goal has the blank in the bottom-right corner: **1 2 3 / 4 5 6 / 7 8 blank**. Start state: **1 2 3 / blank 4 6 / 7 5 8**. Only three tiles are out of place (4, 5 and 8), so the start has h = 1 + 1 + 1 = **3**.

Concept Box: Step by step with the Manhattan values

We slide a tile into the blank. In the notation below a move is named by the direction the **blank** moves.

Step 1. Frontier = {start (h = 3)}. Pop it. Its neighbours:

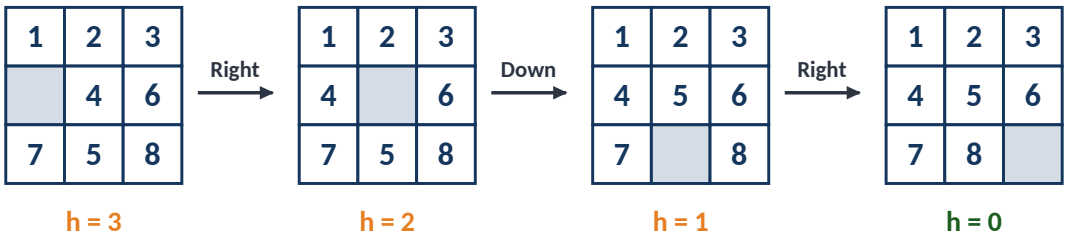
- blank Up: blank 2 3 / 1 4 6 / 7 5 8, h = 4
- blank Down: 1 2 3 / 7 4 6 / blank 5 8, h = 4
- **blank Right** (swap with tile 4): 1 2 3 / 4 blank 6 / 7 5 8, h = **2**

Smallest h is 2, so pop that state.

Step 2. State 1 2 3 / 4 blank 6 / 7 5 8 (h = 2). Neighbours: blank Up h = 3, blank Left h = 3 (that just undoes the last move, so it is skipped as already seen), blank Right h = 3, **blank Down** (swap with tile 5): 1 2 3 / 4 5 6 / 7 blank 8, h = **1**. Pop the h = 1 state.

Step 3. State 1 2 3 / 4 5 6 / 7 blank 8 (h = 1). Neighbours: blank Up h = 2 (seen), blank Left h = 2, **blank Right** (swap with tile 8): 1 2 3 / 4 5 6 / 7 8 blank, h = **0**.

Step 4. Pop the h = 0 state. It is the goal. Done in **3 moves: Right, Down, Right**.



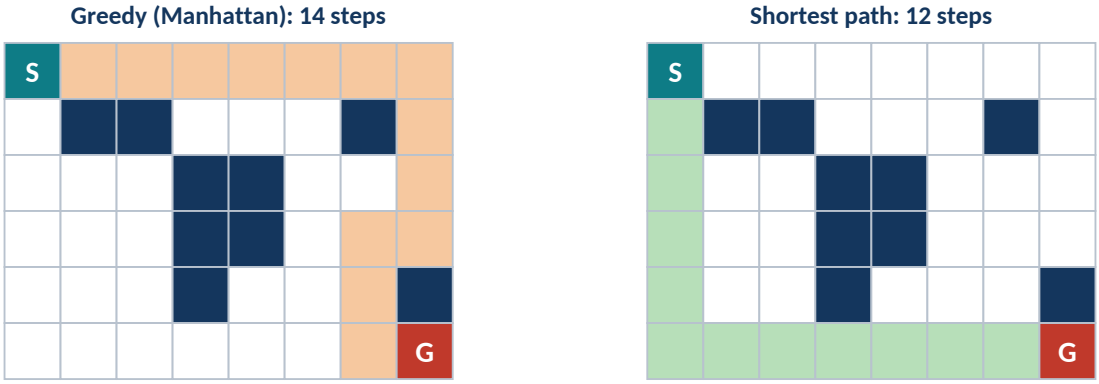
GBFS trace. The h value (Manhattan) falls 3, 2, 1, 0 as the blank moves Right, Down, Right.

Why it worked

In this puzzle every step lowered h by exactly 1, so greedy walked straight to the goal. Greedy shines when the heuristic landscape is smooth and has no traps.

5.5 Worked Example B: GBFS on a Grid With Walls

Now a path-finding grid. **S** is the start (row 0, col 0) and **G** the goal (row 5, col 7). Dark squares are walls. You can move up, down, left or right. The heuristic is the Manhattan distance to G, so h(S) = |0 - 5| + |0 - 7| = **12**.



Left: the route greedy returned. Right: the true shortest route. Tinted squares are the path.

Concept Box: What happened and what to learn from it

- **Greedy** always expands the open cell with the smallest Manhattan distance to G. It expanded **15** cells and returned a path of **14** steps.
- The **shortest** path has **12** steps. Greedy was **2 steps longer**, because it followed squares that looked close to G and committed to a route without checking its cost.
- Notice h = 12 and the best possible path is also 12 here. Manhattan is exactly right when there are no walls, and **underestimates** when walls force detours. That is normal and it is why it is still admissible.
- If you run **A*** on the same grid with the same heuristic, it returns a path of exactly 12 steps, since it also counts g(n).
- Ties (two cells with equal h) are broken by the data structure. Different tie-breaking can give a different path of equal quality or worse, which is another reason greedy is not guaranteed optimal.

5.6 GBFS With Manhattan Distance in Python

Here is the whole algorithm in about 25 lines. It works on a grid stored as a list of strings, where **#** is a wall. Read the comments, then try changing the heuristic to see the effect.

```
import heapq

def manhattan(a, b):
    # a and b are (row, col) tuples
    return abs(a[0] - b[0]) + abs(a[1] - b[1])

def gbfs(grid, start, goal):
    rows, cols = len(grid), len(grid[0])
    frontier = [(manhattan(start, goal), start)]   # priority is h only
    came_from = {start: None}                      # also acts as "seen"
    expanded = 0
    while frontier:
        h, cell = heapq.heappop(frontier)          # lowest h first
        expanded += 1
        if cell == goal:
            path = []
            while cell is not None:                # walk back to the start
                path.append(cell)
                cell = came_from[cell]
            return path[::-1], expanded
        r, c = cell
        for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
            nxt = (r + dr, c + dc)
            if (0 <= nxt[0] < rows and 0 <= nxt[1] < cols
                    and grid[nxt[0]][nxt[1]] != '#' and nxt not in came_from):
                came_from[nxt] = cell
                heapq.heappush(frontier, (manhattan(nxt, goal), nxt))
    return None, expanded                          # no path exists
```

```
grid = ["S.......", ".##...#.", "...##...", "...##...", "...#...#", ".......G"]
path, expanded = gbfs(grid, (0, 0), (5, 7))
print(len(path) - 1, "steps,", expanded, "cells expanded")
print(path)
```

On the grid above this prints **14 steps, 15 cells expanded**. To turn it into A*, change the priority from `manhattan(nxt, goal)` to `cost_so_far[nxt] + manhattan(nxt, goal)`, and keep a `cost_so_far` dictionary of g values.

5.7 Common Mistakes and an Exam Checklist

Concept Box: Before you write your answer

- **Never count the blank** when computing the 8-puzzle heuristic.
- Use **|row gap| + |col gap|**. Do not use a straight line.
- GBFS orders by **h only**. If you add g, you are doing A*.
- Mark visited states, otherwise greedy can loop (that is why it is incomplete without repeated-state checking).
- State the verdict: **GBFS is fast but neither optimal nor guaranteed complete**; Manhattan distance is **admissible and consistent** for 4-direction unit-cost moves.
- Always show the **frontier after every expansion** with the h value next to each node. Examiners give marks for the trace.

PART 6

Beyond Classical Search: Local Search

When the search space is far too big to explore systematically, improve a single answer step by step: hill climbing, simulated annealing, beam search and genetic algorithms.

6.1 Why We Need Something Beyond Classical Search

Watch First

Start with one of these, then come back and read the notes. Links to the Uninformed vs Informed and BFS lectures open the video directly. The others open a Gate Smashers search for that exact lecture title, so tap the top result.

- Gate Smashers: Hill Climbing Algorithm with Real Life Examples | Heuristic Search
- Gate Smashers: Simulated Annealing
- Gate Smashers: Genetic Algorithm

The methods so far (including A*) systematically explore the space, perhaps with smart pruning. The best of them handle roughly **10^{100} states, about 1,000 binary variables**. Some real problems (certain reasoning and planning tasks) have spaces around **$10^{30,000}$** states. Another approach is needed: **iterative improvement**, also called **local search**.

Concept Box: Iterative improvement: the main idea

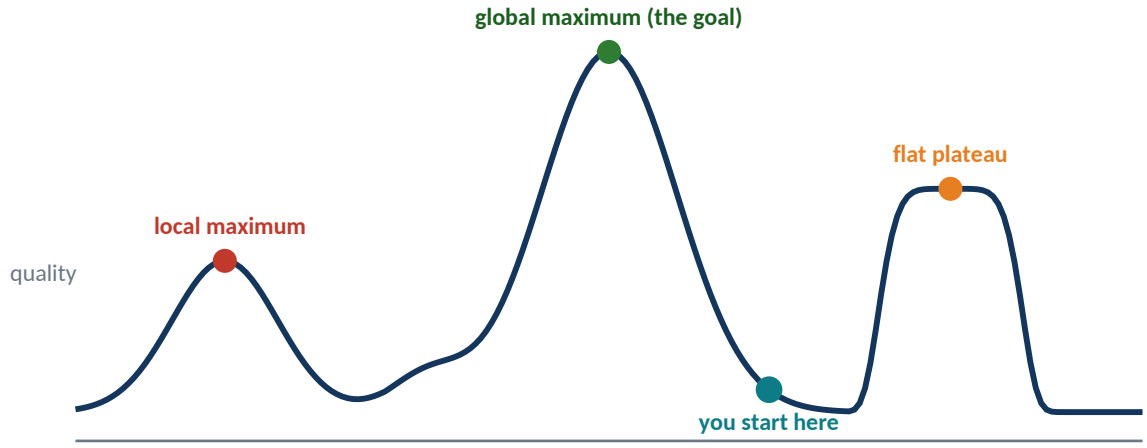
In many optimization problems **the goal state itself is the answer**; the path to it does not matter. The state space is a set of **complete configurations** (a full schedule, a full tour, a full board). The job is to find the best configuration, or just a feasible one.

You need an **evaluation (objective) function** that scores how good each state is. Ideally the closer a state is to an optimal goal, the better its score.

Main idea: start with a **random complete configuration** and keep making small **local changes** that improve its quality. Memory use is tiny, because you only remember the current state.

Concept Box: The landscape picture

Imagine every state as a point on a surface whose height is its quality. Search means moving along the surface looking for the **highest peak** (or, if you minimise cost, the **lowest valley**). Peaks are optimal solutions.



The state-space landscape. A good local search should reach the green peak, but it may end up on the red one or wander on the orange plateau.

Two classic problems

Concept Box: The Travelling Salesperson Problem (TSP)

Visit every city once and return home with the **shortest tour**. Score h = **length of the tour**.

Strategy: start with any complete tour, then repeatedly perform **pairwise exchanges** (swap the order of two cities, or reverse a segment) that shorten it. Variants of this get within about **1 percent of optimal** for tens of thousands of cities, very quickly.

Concept Box: The n-queens problem

Place **n queens** on an n x n board so that no two share a **row, column or diagonal**. Score h = **number of conflicts** (attacking pairs).

Strategy: move a queen to reduce the number of conflicts. On the slide, a move takes h from **5 down to 2**.

It almost always solves n-queens almost instantly even for **very large n, such as n = 10^6**.

6.2 Hill-Climbing Search

Also called **gradient ascent or descent**. The slide's joke: it is like **climbing Everest in thick fog with amnesia**. You feel which neighbouring step goes uphill, take it, and forget where you were.

```
function HILL-CLIMBING(problem) returns a state that is a local maximum
    current = MAKE-NODE(INITIAL-STATE[problem])
    loop:
        neighbor = a highest-valued successor of current
        if VALUE[neighbor] < VALUE[current]: return STATE[current]
        current = neighbor
```

In words: look at all neighbours, jump to the best one, and stop when no neighbour is better. (The slide uses "<", so it keeps moving across equal values; many textbooks use "less than or equal" to stop on flat ground.)

Concept Box: Shortcomings of hill climbing

- **Local maximum:** a peak higher than all neighbours but lower than the true best. You stop on it, and depending on the **initial state** you may get stuck.
- **Plateau:** a flat region. A **"flat" local max** has no way up; a **shoulder** is flat now but has a way up further on. A plain climber can wander forever.
- **Slow convergence:** it may take many tiny steps.
- **Step size:** in continuous spaces, choosing the step size is non-trivial.

Reassurance from the slides: in very high-dimensional spaces, true local optima are surprisingly rare. There is often an escape direction to a better state.

Concept Box: Improvements to hill climbing

- **Random restart:** run it many times from random starting points and keep the best result.
- **Look ahead:** expand up to m > 1 generations before choosing the best node.
- **Add noise:** sometimes allow a downhill move (this leads to simulated annealing).
- **Keep n > 1 nodes** at each step (this leads to beam search).

6.3 Simulated Annealing

The idea comes from heating metal and cooling it slowly so it settles into a strong structure. **Simulated annealing improves hill climbing by allowing occasional downhill moves** to reduce the chance of getting stuck on a local maximum.

```
function SIMULATED-ANNEALING(problem, schedule) returns a solution state
    current = MAKE-NODE(INITIAL-STATE[problem])
    for t = 1 to infinity:
        T = schedule[t]                       # the "temperature"
        if T = 0 or IS-GOAL-STATE(current): return current
        next = a randomly selected successor of current
        deltaE = VALUE[next] - VALUE[current]
        if deltaE > 0: current = next         # better: always accept
        else: current = next only with probability e^(deltaE / T)
```

Concept Box: How the acceptance probability behaves (with numbers)

A better move is always taken. A worse move ($\Delta E < 0$) is taken with probability $\mathbf{e}^{-|\Delta E| / T}$, written on the slide as $(1/e)^{|\Delta E|/T}$.

- The **bigger the drop** $|\Delta E|$, the **less likely** it is accepted.
- The **higher the temperature T**, the **more likely** it is accepted.

Numbers: a bad move with $\Delta E = -2$. At $T = 10$ the probability is $e^{-0.2} \approx$ **0.82** (you will usually accept it, so the search explores freely). At $T = 1$ it is $e^{-2} \approx$ **0.14**. As T approaches 0 it becomes almost 0, and the algorithm behaves like plain hill climbing.

Why it works: early on (hot) it wanders and can escape local maxima; later (cold) it settles into a good peak. The rule that lowers T over time is the **annealing schedule**.

- **Guarantee:** if the schedule lowers T slowly enough, the search converges to the best state. The slides call this "not necessarily an interesting guarantee", because "slowly enough" may take impractically long. In practice you tune the schedule by trying it on test problems.

- **History and use:** devised by Metropolis et al. in 1953 to model physical processes. Widely used in VLSI chip layout, airline scheduling and similar problems.

> **Memory Hook**
>
> Hot = adventurous (accepts bad moves). Cold = careful (hill climbing). Anneal slowly and you end up in a good place.

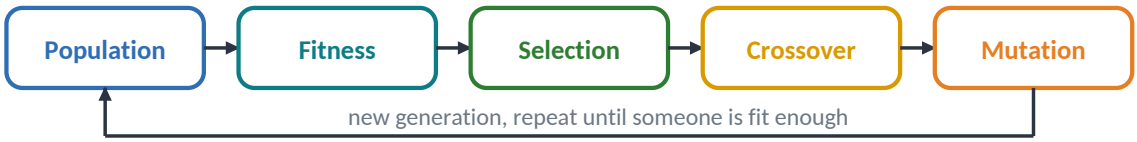
6.4 Local Beam Search

Instead of one state, keep **k states**. At each step generate all their successors and keep the **best k of all of them**.

> **Concept Box: Beam search is not "k parallel hill climbs"**
>
> In k independent searches each one lives or dies alone. In beam search, **the searches share information**: if one search finds a good state, its successors can crowd out the others, so good searches **recruit** the rest to join them.
>
> **Problem:** often all k states end up on the same local peak and the diversity is lost.
>
> **Fix (stochastic beam search):** choose the k successors **randomly, biased toward the better ones**, instead of strictly taking the top k.

6.5 Genetic Algorithms (GA)

Inspired by Darwin's natural selection. A **population** of individuals evolves: the fit ones are selected and reproduce, and over generations the population gets fitter. A GA is basically a **stochastic local beam search**, except new states come from **pairs of parents** rather than from one state.



The genetic algorithm loop.

```
function GENETIC-ALGORITHM(population, FITNESS-FN) returns an individual
    repeat
        parents    = SELECTION(population, FITNESS-FN)
        population = REPRODUCTION(parents)
    until some individual is fit enough
    return the best individual in population, according to FITNESS-FN
```

- **Reproduction** (crossover and mutation) creates diversity and novelty.
- **Selection** reduces diversity but pushes quality up by increasing fitness.
- Local maxima are avoided because even **low-scoring individuals have a non-zero chance** of being selected.
- Over the generations, the population clusters around the high peaks of the fitness landscape (the slide's generation 0 versus generation N picture).

Concept Box: The classic GA, using 8-queens (the slide example)

Each state is a **string over a finite alphabet**, and each character is a **gene**. Here a state is 8 digits: digit i gives the row of the queen in column i, such as **24748552**.

- **(a) Initial population:** 24748552, 32752411, 24415124, 32543213.
- **(b) Fitness function:** number of non-attacking pairs. The scores are 24, 23, 20 and 11.
- **(c) Selection:** probability proportional to fitness. The total is 24 + 23 + 20 + 11 = 78, so the chances are 24/78 = **31%**, 23/78 = **29%**, 20/78 = **26%**, 11/78 = **14%**. Fitter strings are picked more often, but weak ones still have a chance.
- **(d) Crossover:** pick a crossover point, then swap tails. Parents 327|52411 and 247|48552 produce children **32748552** and **24752411**.
- **(e) Mutation:** randomly change a gene or two, for example change one digit of a child, to add fresh variation.

Concept Box: Choosing an encoding: the secret ingredient

Choosing how to represent states as strings is crucial. **Crossover helps only if substrings are meaningful building blocks** that can be recombined into new meaningful configurations.

Possible encodings: **character strings** (0101...1100), **sequences of real numbers** (43.2, -33.1, ..., 89.2), **tuples of elements**, **lists of rules**, and even **program elements** (genetic programming).

Remember: **GAs are not evolution.** Real genes encode the machinery that builds the body, and a GA's strings are far simpler.

Concept Box: Two more encoding examples from the slides

Find the location closest to several given cities. Encode a location as a 16-bit string, such as 1001010101011100: the first 8 bits are the X-coordinate and the last 8 bits are the Y-coordinate. Fitness is the median distance to each city. Crossover takes the X-coordinate from one parent and the Y-coordinate from the other. Mutation flips one or more bits.

Find the maximum of a function. Encode a state as a vector of n numbers. Combination has several options, and mutation randomly replaces a value in the vector.

Concept Box: Recombination for number vectors

Discrete recombination is like crossover: each child parameter is taken from either parent with equal probability. Example: parents (8, 12, 31, ..., 5) and (2, 5, 23, ..., 14) might give (**2**, 12, 31, ..., **14**).

Intermediate recombination: each child value is the **average** of the parents' values. The same parents give (5, 8.5, 27, ..., 9.5), since (8+2)/2 = 5, (12+5)/2 = 8.5, (31+23)/2 = 27 and (5+14)/2 = 9.5.

Concept Box: GA for the Travelling Salesperson Problem

Representation: an **ordered list of city numbers** (an order-based GA). With cities 1 London, 2 Venice, 3 Iowa City, 4 Singapore, 5 Beijing, 6 Phoenix, 7 Tokyo, 8 Victoria, a tour is such as **CityList1 = (3 5 7 2 1 6 4 8)**.

Why ordinary crossover fails: swapping tails of two tours can visit a city twice and skip another. Hence **Order 1 crossover**:

- **Step 1:** copy a randomly chosen portion of Parent 1 into the Child, in the same positions.
- **Step 2:** fill the remaining blanks with the cities from Parent 2, going left to right and **skipping duplicates**.

Worked example. Parent 1 = (3 5 7 2 1 6 4 8), Parent 2 = (2 5 7 6 8 1 3 4). Copy the middle portion (7 2 1 6) from Parent 1: Child = (_ _ 7 2 1 6 _ _). Read Parent 2 left to right: 2 (already used), 5 (use it), 7 (used), 6 (used), 8 (use it), 1 (used), 3 (use it), 4 (use it). The blanks are filled with 5, 8, 3, 4 giving Child = **(5 8 7 2 1 6 3 4)**.

Mutation: swap two cities in the list. Before (5 8 **7** 2 1 **6** 3 4), after (5 8 **6** 2 1 **7** 3 4).

Memory Hook: the local search family

GA recipe: Population, Fitness, Selection, Crossover, Mutation, repeat. Local search family tree: Hill climbing, add noise gives Simulated annealing, keep k states gives Beam search, breed the k states gives Genetic algorithm.

PART 7

Adversarial Search: Playing Games

Search when an opponent fights back. Minimax, alpha-beta pruning and evaluation functions.

7.1 Games Versus Ordinary Search

Watch First

Start with one of these, then come back and read the notes. Links to the Uninformed vs Informed and BFS lectures open the video directly. The others open a Gate Smashers search for that exact lecture title, so tap the top result.

- Gate Smashers: Introduction to Game Playing in Artificial Intelligence
- Gate Smashers: Minimax Algorithm in Game Playing
- Gate Smashers: Alpha Beta Pruning

In ordinary search, you control every decision. In a game, **an opponent controls some of them**, and you cannot predict which. Two consequences from the slides:

- **"Unpredictable" opponent:** your solution must specify a move for **every possible opponent reply**. This is a **contingency problem**.
- **Time limits:** you are unlikely to search all the way to the end of the game, so you must **approximate**.

Game-playing programs (chess, checkers) have been built since the 1950s. We assume a **rational opponent** who tries to maximise their own utility (payoff), which usually means minimising ours.

Concept Box: A game as a search problem

- **Initial state:** the starting board position and whose turn it is.
- **Operators (actions):** the legal moves a player can make.
- **Terminal test:** is the game over?
- **Utility (payoff) function:** a number describing the outcome, for example +1 win, 0 draw, -1 loss.

Search objective: find the sequence of moves that maximises your utility, **taking the opponent's moves and their utility into account**.

The structure is a **game tree** for a 2-player, deterministic, turn-taking game. Levels alternate between **MAX** (you, wanting a high value) and **MIN** (opponent, wanting a low value). A single move by one player is a **ply**.

7.2 The Minimax Algorithm

Assume the opponent always plays the best reply against you. Then **minimax** finds your best move. The steps from the slides:

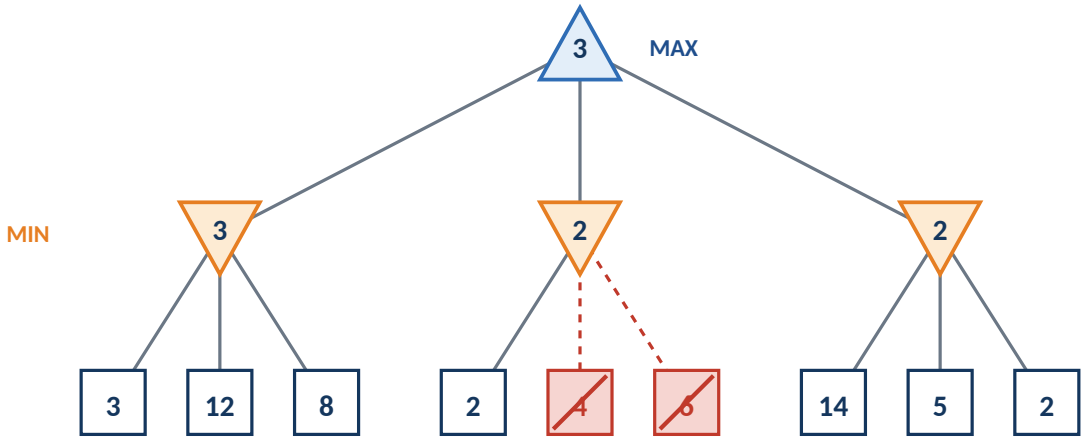
- **Generate** the game tree.
- **Evaluate** the terminal states using the utility function.
- **Propagate values upward:** at a MAX node take the **maximum** of the children, at a MIN node take the **minimum**.

Concept Box: A 2-ply example you can redo (and what the numbers mean)

MAX moves first and has three choices. After each, MIN has three replies. The leaf utilities are (3, 12, 8), (2, 4, 6) and (14, 5, 2).

- MIN picks the **smallest** in each group: min(3, 12, 8) = **3**, min(2, 4, 6) = **2**, min(14, 5, 2) = **2**.
- MAX then picks the **largest** of 3, 2, 2, which is **3**. So MAX chooses the first move.

Meaning of the root value 3: it is the best payoff you can guarantee against a perfect opponent. Move 2 looked tempting because of the 14 and the 4, but a smart MIN would never allow that.



Red leaves are pruned by alpha-beta: they are never even looked at

Game tree for the example. Triangles pointing up are MAX nodes, pointing down are MIN nodes. The root value is 3.

Concept Box: Minimax properties

Perfect play for deterministic, two-player games: it picks the move with the highest minimax value, the best achievable payoff against best play.

Time: $O(b^m)$. **Space:** O(bm), since it runs depth-first. For chess, b is about 35 and m about 100, so exact search is hopeless. We need a shortcut: **alpha-beta pruning**.

7.3 Alpha-Beta Pruning

Idea: **some branches will never be played by rational players**, because they contain bad decisions for one side. If we can prove a branch will not change the final choice, we skip it entirely.

Concept Box: Alpha and beta in plain words

- **Alpha (α)** = the best (highest) value **MAX** is already guaranteed so far on the current path.
- **Beta (β)** = the best (lowest) value **MIN** is already guaranteed so far on the current path.

Pruning rules: at a **MIN** node, if its value falls to **α or below**, stop: MAX already has something better elsewhere and will never come here. At a **MAX** node, if its value rises to **β or above**, stop: MIN will never allow this branch.

```
function ALPHA-BETA-SEARCH(state) returns an action
    v = MAX-VALUE(state, -inf, +inf)
    return the action in ACTIONS(state) with value v

function MAX-VALUE(state, alpha, beta):          # called for MAX nodes
    if TERMINAL-TEST(state): return UTILITY(state)
    v = -inf
    for each action a in ACTIONS(state):
        v = max(v, MIN-VALUE(RESULT(state, a), alpha, beta))
        if v >= beta: return v                    # prune the rest
        alpha = max(alpha, v)                     # update best for MAX
    return v

function MIN-VALUE(state, alpha, beta):          # called for MIN nodes
    if TERMINAL-TEST(state): return UTILITY(state)
    v = +inf
    for each action a in ACTIONS(state):
        v = min(v, MAX-VALUE(RESULT(state, a), alpha, beta))
        if v <= alpha: return v                   # prune the rest
        beta = min(beta, v)                       # update best for MIN
    return v
```

Concept Box: The same example with alpha-beta bookkeeping

- **First MIN node**, leaves 3, 12, 8: after seeing 3 its value is at most 3, and 12 and 8 do not lower it, so it returns **3**. The root now has **α = 3**.
- **Second MIN node** (α = 3): the first leaf is **2**, so this node is at most 2, which is already ≤ α. MAX will never pick it. **Prune** the leaves 4 and 6.
- **Third MIN node** (α = 3): leaves 14, 5, 2 give values 14, then 5, then 2. It returns **2** after the last leaf (the 2 is ≤ α, but there was nothing left to skip).
- **Root** = max(3, 2, 2) = **3**, the same answer as plain minimax, with **two leaves never examined**.

Concept Box: Properties of alpha-beta

- **Pruning does not change the final result.** It only saves work.
- **Move ordering matters:** looking at good moves first prunes more.
- With **perfect ordering**, time drops to **$O(b^{m/2})$**, which **doubles the search depth** you can afford in the same time.
- It is a simple example of **metareasoning**: reasoning about which computations are even relevant.

7.4 Evaluation Functions and Real Games

Even with pruning, real games are too deep to search to the end. So we **stop at a depth limit** and score the unfinished position with an **evaluation function** that estimates its value (a heuristic, just like h(n)).

Concept Box: Designing an evaluation function

A common form is a **weighted sum of features**: $\text{Eval}(s) = w_1 f_1(s) + w_2 f_2(s) + \ldots + w_n f_n(s)$.

Chess example: features are the number of each piece type, with weights such as pawn 1, knight 3, bishop 3, rook 5, queen 9. A position with an extra rook scores about +5 for that side. Other features could be king safety or pawn structure.

A good evaluation function should be **fast to compute**, agree with the true utility at terminal states, and **correlate strongly with the real chances of winning**.

The slide on "further extensions to real games" has only a title, so for completeness, the usual extras are: cutting off search at a depth limit, using **iterative deepening** with a time limit, storing repeated positions, opening and endgame databases, and handling **chance** (dice) with expected values.

PART 8

Last-Day Revision Sheet

Everything squeezed onto a few pages. Read this the night before the exam, after you have understood the earlier parts.

8.1 Search Algorithms Side by Side

Uninformed

- **BFS:** queue. Complete if b finite. Optimal if step cost equal. Time and space $O(b^{d+1})$ on the slide (about b^d in many textbooks).
- **UCS:** priority queue by g. Complete if step cost ≥ ε. Optimal. $O(b^{\text{ceil}(C^*/\varepsilon)})$.
- **DFS:** stack. Not complete, not optimal. Time $O(b^m)$, space O(bm).
- **DLS:** DFS with limit l. Complete if d ≤ l. Not optimal. Time $O(b^l)$, space O(bl).
- **IDS:** repeats DLS with l = 0, 1, 2, … Complete. Optimal if step cost 1. Time $O(b^d)$, space O(bd).
- **Bidirectional:** meet in the middle. About $O(b^{d/2})$. Needs predecessors.

Informed

- **Greedy best-first:** f = h. Not complete (without repeated-state check), not optimal. Time and space $O(b^m)$.
- **A*:** f = g + h. Complete, optimal if h admissible (tree search) or consistent (graph search). Space is the problem.
- **RBFS:** A*-like in linear space using f_limit and backed-up values. Re-expands nodes.

Local search

- **Hill climbing:** move to the best neighbour. Gets stuck on local maxima, plateaus.
- **Simulated annealing:** accept bad moves with probability $e^{-|\Delta E|/T}$; cool T slowly.
- **Local beam:** keep k states, share information. **Stochastic beam:** random choice biased to the good.
- **Genetic algorithm:** population, fitness, selection, crossover, mutation.

Adversarial

- **Minimax:** MAX takes max, MIN takes min. Time $O(b^m)$, space O(bm).
- **Alpha-beta:** same answer, skips branches. Best case $O(b^{m/2})$.
- **Evaluation function:** estimates value at the depth cutoff.

8.2 Formulas to Memorise

Write these from memory

- Agent function: f : P* → A. Agent = architecture + program.

- Rational agent: picks the action with the **maximum expected performance measure** given percepts and knowledge.
- Utility: U : State → real numbers.
- Best-first: f(n) = h(n) for greedy, f(n) = g(n) + h(n) for A*.
- Admissible: h(n) ≤ true cost to goal. Consistent: h(n) ≤ c(n, a, n') + h(n').
- Manhattan distance: |r1 - r2| + |c1 - c2|. For the 8-puzzle, sum over all tiles except the blank.
- Effective branching factor: $N + 1 = 1 + b^* + (b^*)^2 + \ldots + (b^*)^d$.
- Simulated annealing acceptance: $e^{-|\Delta E|/T}$.
- Roulette selection in a GA: fitness of one / total fitness.

8.3 Mnemonics Collected in One Place

- **PEAS:** Performance, Environment, Actuators, Sensors.
- **Environment questions (ODESDA):** Observable, Deterministic, Episodic, Static, Discrete, Agents. "Our Dog Eats Sausages, Dozing Alone".
- **Agent types (RMGUL):** Reflex, Model, Goal, Utility, Learning. "Really Mighty Goats Use Legs".
- **Four AI approaches:** Think humanly (cognitive science), Act humanly (Turing), Think rationally (logic), Act rationally (agent).
- **Search judging:** Complete, Optimal, Time, Space (C-O-T-S).
- **BFS vs DFS:** Queue and Broad vs Stack and Deep.
- **Greedy vs A*:** hope only (h) vs history plus hope (g + h).
- **GA loop:** Population, Fitness, Selection, Crossover, Mutation.

8.4 Likely Exam Questions With Short Answers

Q1. Define a rational agent.

An agent that, for each percept sequence, selects the action expected to maximise its performance measure given the percept sequence and its built-in knowledge.

Q2. Why is rational not the same as omniscient?

Percepts may not reveal everything and outcomes may differ from expectations, so a rational choice can still end badly. Rationality is about the decision, not the result.

Q3. What does PEAS stand for? Give the PEAS for an automated taxi.

Performance measure, Environment, Actuators, Sensors. Taxi: safe, fast, legal, comfortable, profitable; roads, traffic, pedestrians, customers; steering, accelerator, brake, horn; cameras, GPS, speedometer, sonar.

Q4. Explain fully observable, deterministic, episodic, static, discrete, single-agent with an example of each.

See Section 2.5: checkers (observable), chess pawn move (deterministic), defective part inspection (episodic), crossword (static), checkers board (discrete), crossword alone (single-agent).

Q5. Compare simple reflex and goal-based agents.

Reflex agents map the current percept to an action using condition-action rules and have no memory or foresight. Goal-based agents keep state, hold goals and consider the future consequences of actions.

Q6. Compare BFS and DFS.

BFS expands shallowest first with a queue, is complete and optimal for equal costs but uses exponential memory. DFS expands deepest first with a stack, uses linear memory, but is neither complete in infinite spaces nor optimal.

Q7. Why is iterative deepening a good choice?

It combines the completeness and (unit-cost) optimality of BFS with the linear memory of DFS, and the repeated work costs little because most nodes are at the deepest level.

Q8. Differentiate greedy best-first search and A*.

Greedy uses f = h and ignores cost so far, so it is fast but neither optimal nor complete. A* uses f = g + h and is complete and optimal with an admissible (consistent) heuristic.

Q9. What is an admissible heuristic? Give an example.

One that never overestimates the true cost to the goal. Straight-line distance for route finding, or Manhattan distance for the 8-puzzle.

Q10. Compute Manhattan distance for tiles at (2,2) going to (0,1).

$|2 - 0| + |2 - 1| = 3$.

Q11. Why does h2 (Manhattan) dominate h1 (misplaced tiles)?

Every misplaced tile contributes at least 1 to the Manhattan sum, so $h2(n) \geq h1(n)$ for all n, and both are admissible. A* with h2 never expands more nodes.

Q12. Explain local maxima and how to escape them.

A local maximum is a state better than all its neighbours but not the best overall. Use random restarts, simulated annealing (accept some bad moves), beam search or genetic algorithms.

Q13. How does simulated annealing differ from hill climbing?

It sometimes accepts worse moves with probability $e^{-|\Delta E|/T}$, with T decreasing over time, so it can escape local maxima.

Q14. Describe the steps of a genetic algorithm.

Create a population, evaluate fitness, select parents in proportion to fitness, apply crossover and mutation to make children, form a new population and repeat until a fit individual is found.

Q15. What does alpha-beta pruning achieve?

It returns the same minimax value while skipping branches that cannot influence the decision. With perfect ordering it reduces time to $O(b^{m/2})$, allowing about double the depth.

Final Word

You now have the whole syllabus in one place. Next step: close the notes, pick any Concept Box title, and explain it from memory. Good luck with the exam!

F I N